

Lógica y Representación 1

Ciclos

Ingeniería de Sistemas
Universidad de Antioquia

Método de Polya - Resumen



PROBLEMA



ALGORITMO



1

Entender el problema

2

Diseñar un plan

3

Implementar el plan

4

Revisar el plan

Entrada, procesamiento y salida

Los programas de computadora realizan estos tres pasos:

1. Se recibe la **entrada**.
2. Se realiza un **proceso** sobre la entrada.
3. Se genera la **salida**



Inputs



Process

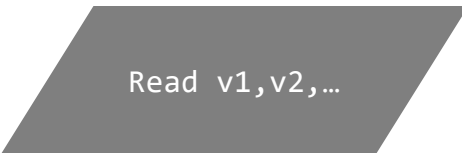
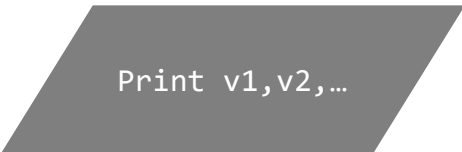


Outputs



Tabla resumen

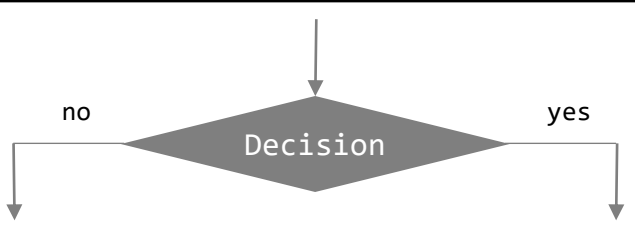
La siguiente tabla muestra la representación equivalente de las piezas esenciales para construir código secuencial

Tipo	Diagrama de bloques	Pseudocodigo	Python
Entrada 		Leer(v1,v2,...)	<pre>v1 = input(...) v2 = input(...)</pre>
Salida 		Escribir(v1,v2,...)	<pre>print(v1,v2,...)</pre>
Proceso 		$c = a + b$	$c = a + b$

En Python, `input()` siempre devuelve texto (`str`); si el valor se va a usar en una operación numérica, debe convertirse con `int()` o `float()` (por ejemplo: `v1 = float(input(...))`).

Tabla resumen

La siguiente tabla, agrega la nueva el nuevo bloque que permite implementar toma de decisiones en los programas.

Diagrama de bloques	Pseudocodigo	Python
	Si (condicion) Entonces instrucciones verdad Sino instrucciones falsa Fin_Si	<pre>if (condicion): instrucciones verdad else: instrucciones falso</pre>

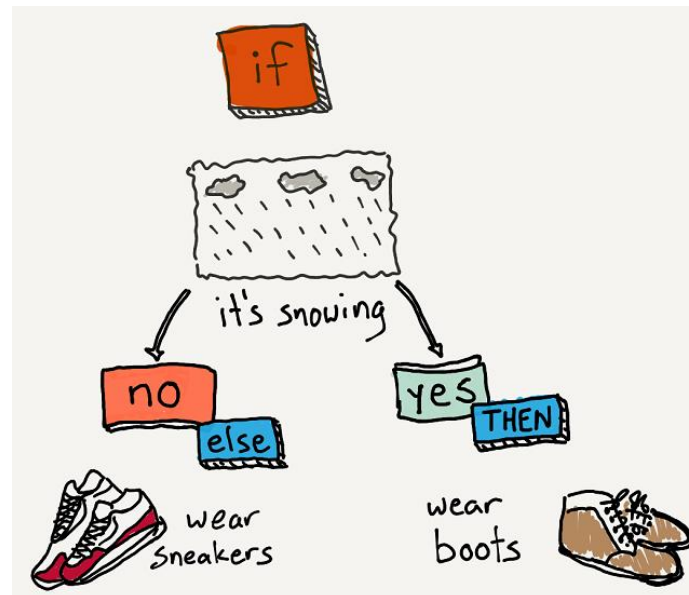


Imagen tomada de CS Intro ([link](#))

Casos en los problemas de decisión lógica

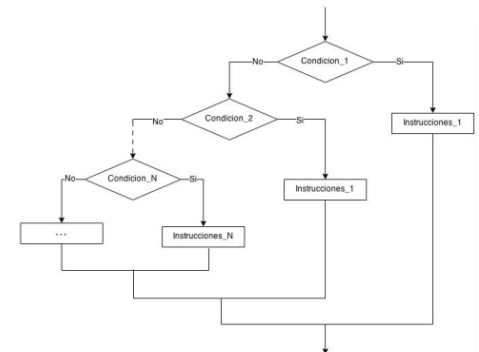
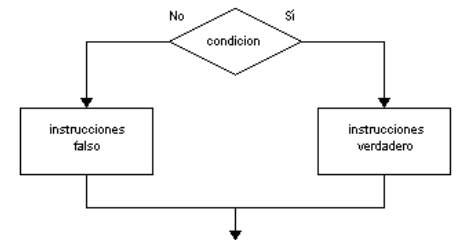
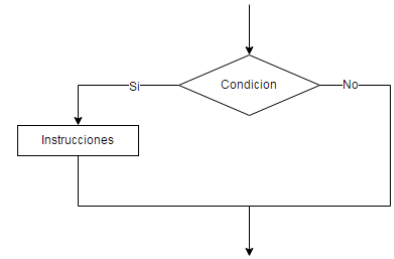


**Alternativas en los
problemas de decisión
lógica**

Alternativa simple

Alternativa doble

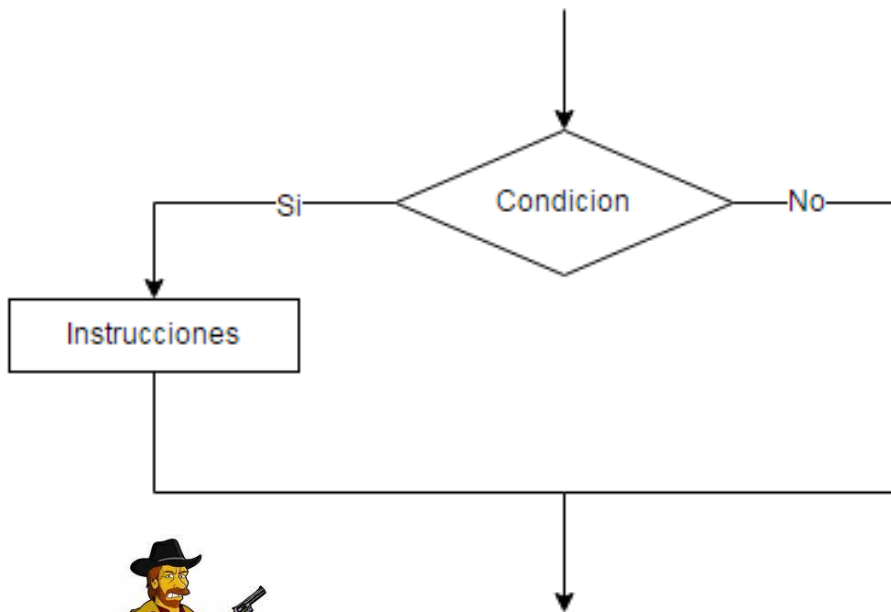
Alternativa múltiple



Alternativa simple

En los problemas de este tipo la condición **falsa** no posee instrucciones.

Diagrama de flujo



Pseudocódigo

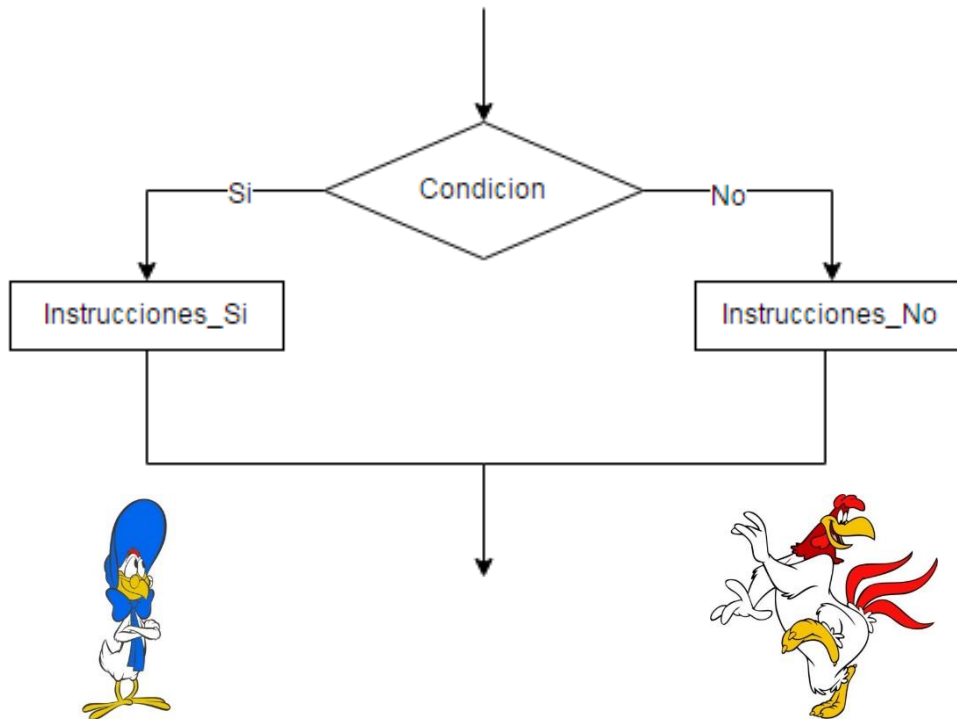
```
Si (condicion) Entonces  
    instrucciones condicion verdadera  
Fin_Si
```

Python

```
if (condicion):  
    instrucciones condicion verdadera
```

Alternativa doble

En los problemas de este tipo, ambos casos (**Falso** y **Verdadero**) poseen instrucciones asociadas.



Pseudocódigo

```
Si (condicion) Entonces  
    instrucciones condicion verdadera  
Sino  
    instrucciones condición falsa  
Fin_Si
```

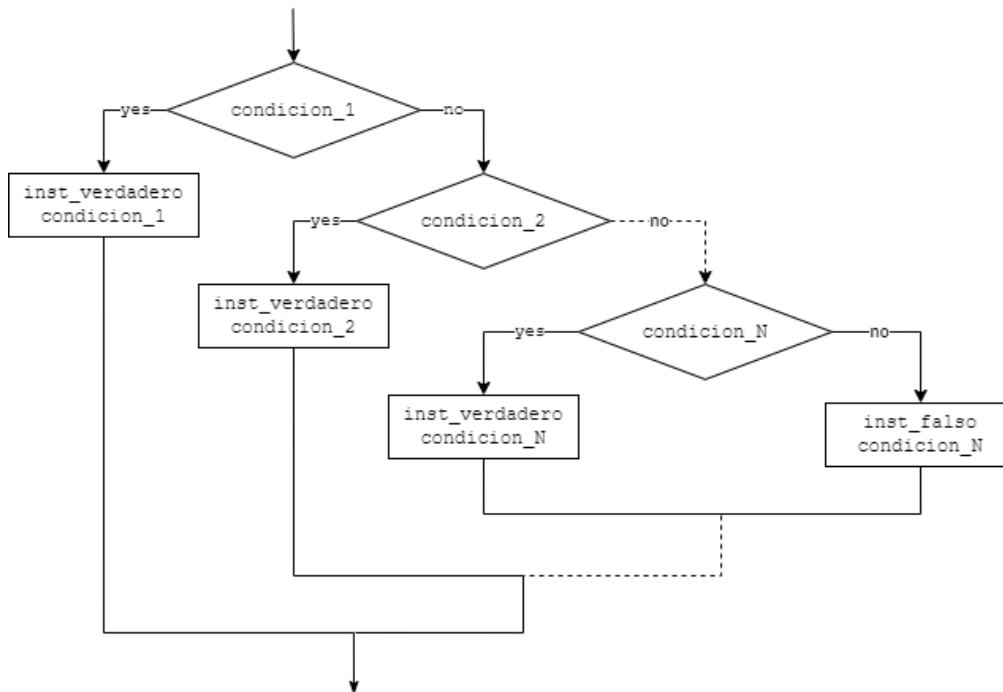
Python

```
if (condicion):  
    instrucciones condicion verdadera  
else:  
    instrucciones condición falsa
```

Alternativa múltiple – Mismo criterio

Veamos el caso en el que todas las condiciones evalúan el **mismo criterio**, por ejemplo, distintos rangos de una misma variable (como una escala de notas)

Diagrama de flujo



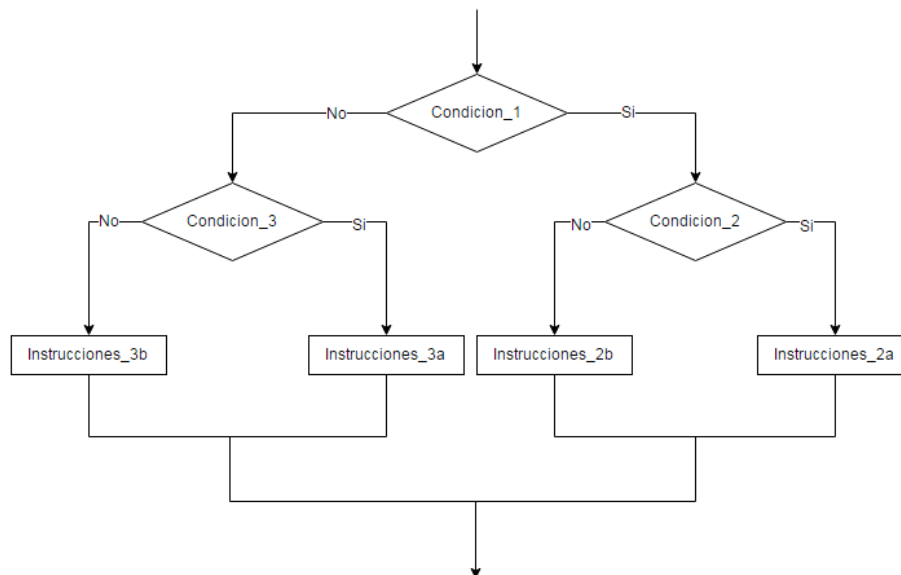
Pseudocódigo

```
Si (condicion_1) Entonces  
    condicion_1 verdad  
Sino  
    Si (condicion_2) Entonces  
        condicion_2 verdad  
    Sino  
        Si (condicion_3) Entonces  
            condicion_3 verdad  
        Sino  
            ...  
            Si (condicion_n) Entonces  
                condicion_n verdad  
            Sino  
                condicion_n falso  
        Fin_Si  
        ...  
    Fin_Si  
Fin_Si
```

Alternativa múltiple – Diferente criterio

Veamos el caso en el que cada rama evalúa un **criterio distinto**, por ejemplo, un umbral que cambia según la decisión anterior (como el sexo y luego un límite de edad diferente para cada uno).

Diagrama de flujo



Pseudocódigo

```
Si (condicion_1) Entonces
  Si (condicion_3) Entonces
    condicion_3a verdad
  Sino
    condicion_3b
  Fin_Si
Sino
  Si (condicion_2) Entonces
    condicion_2a
  Sino
    condicion_2b
  Fin_Si
Fin_Si
```

Ejemplo de repaso

Ejemplo – Año bisiesto:

Se requiere un algoritmo que al ser preguntado por un año cualquiera, diga si éste es un año bisiesto o no.



1892	1920	1944	1968	1992
1896	1924	1948	1972	1996
1904	1928	1952	1976	2000
1908	1932	1956	1980	2004
1912	1936	1960	1984	2008
1916	1940	1964	1988	2012

Un **año bisiesto** es el divisible entre 4, salvo que sea año secular —último de cada siglo, terminado en «00»—, en cuyo caso también ha de ser divisible entre 400.

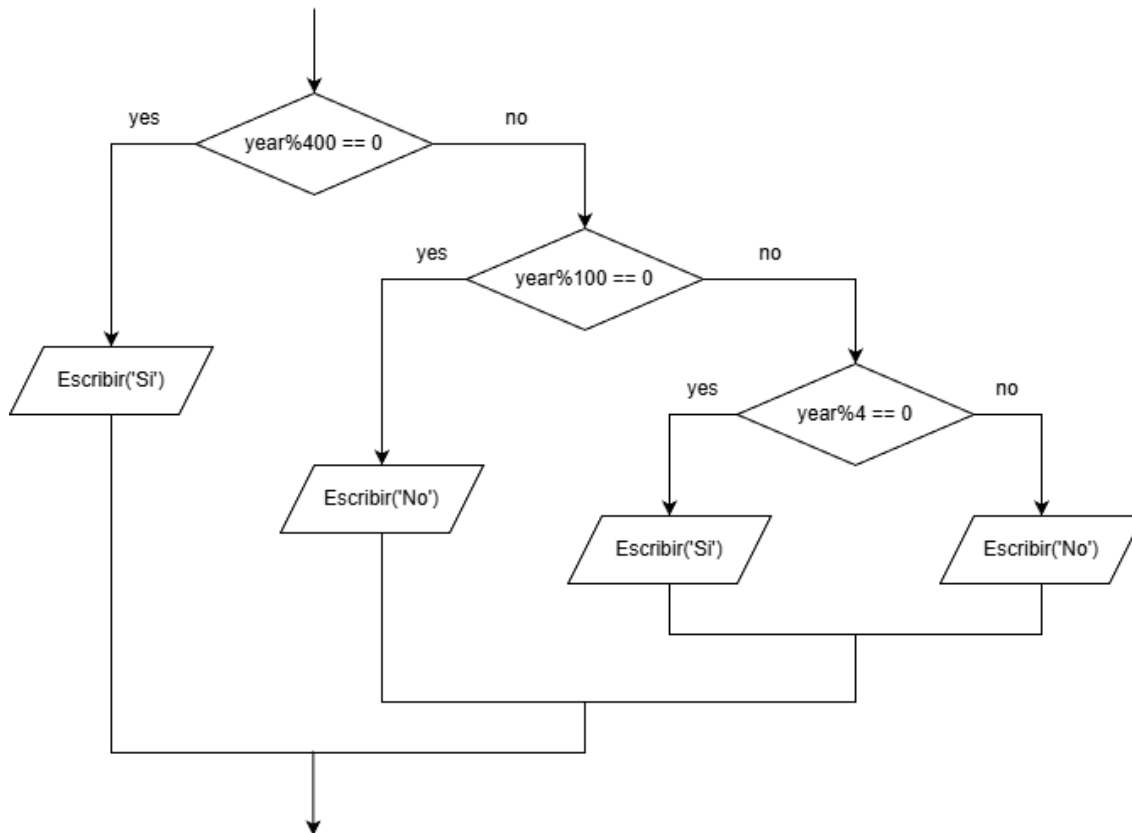
Para profundizar mas ver esto en [wikipedia](https://es.wikipedia.org/wiki/A%C3%B1o_bisiesto)

Ejemplo de repaso

Según la definición anterior para año bisiesto tenga en cuenta que:

- Debe ser múltiplo de cuatro pero no multiplo de 100 (fin de siglo)
- Si es fin de siglo, también deber ser multiplo de 400.

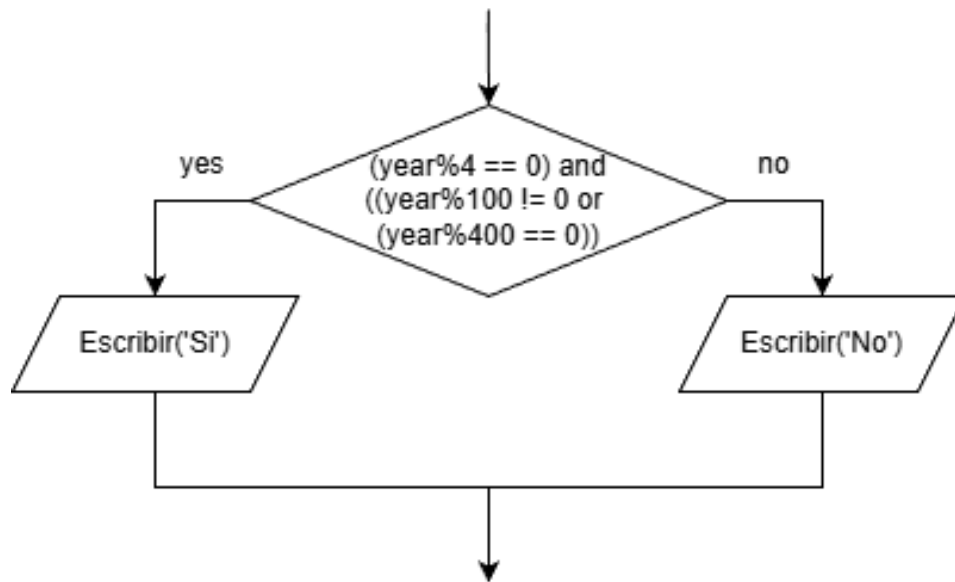
year	Bisiesto
2023	No
2024	Si
1900	No
2000	Si



Ejemplo de repaso

Según la definición anterior para año bisiesto tenga en cuenta que:

- Debe ser múltiplo de cuatro pero no múltiplo de 100 (fin de siglo)
- Si es fin de siglo, también deber ser múltiplo de 400.



year	Bisiesto
2023	No
2024	Si
1900	No
2000	Si

Ejemplo de repaso

Ejemplo – Año bisiesto:



Datos de entrada:

- Año

Datos de salida:

- Decir si el año es bisiesto o no

Definición de variables:

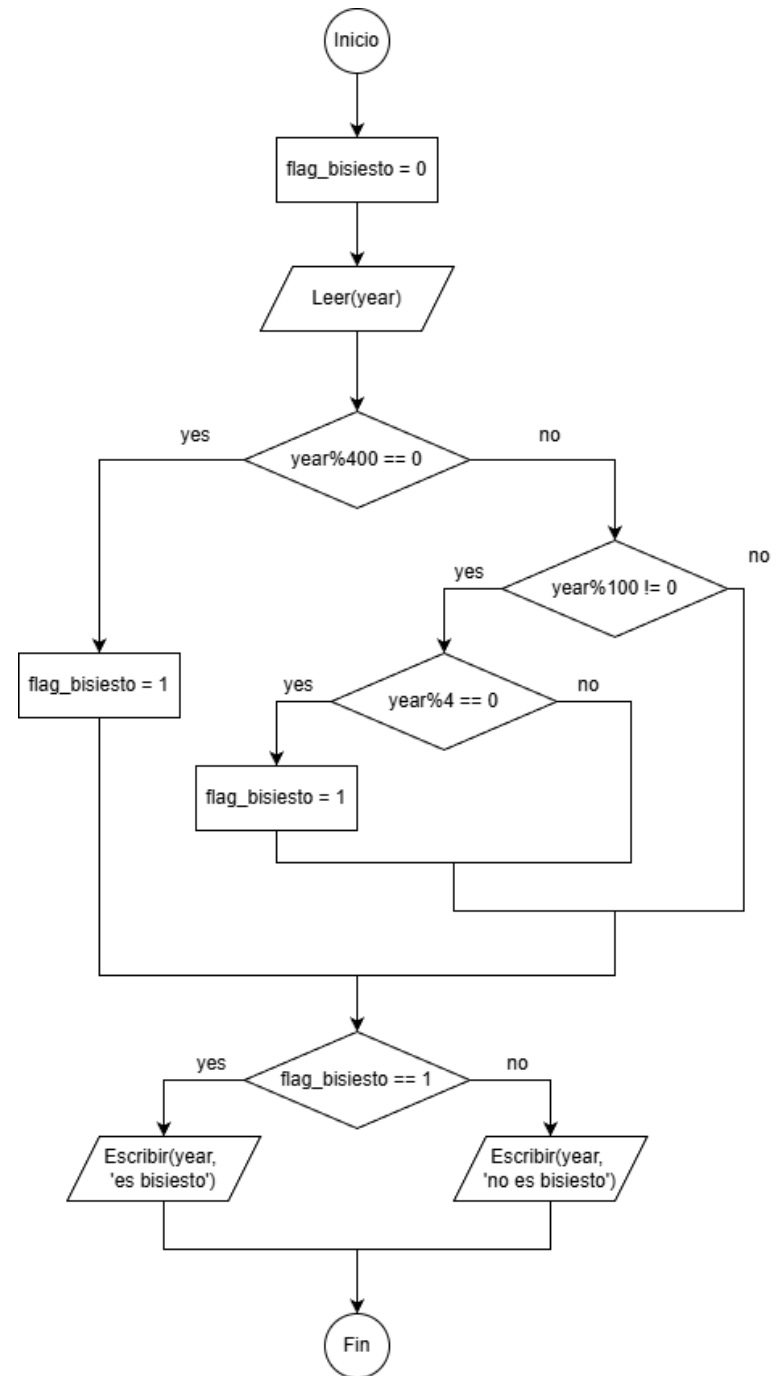
- **Entrada/salida:**
 - `year`: Año.
- **Auxiliares:**
 - `flag_bisiesto`: Bandera que indica si el año es bisiesto o no.

Ejemplo de repaso

Ejemplo – Año bisiesto:

Pseudocódigo

```
Inicio
  flag_bisiesto = 0
  Leer(year)
  Si(year%400 == 0) Entonces
    flag_bisiesto = 1
  Sino
    Si(year%100 != 0) Entonces
      Si(year%4 == 0) Entonces
        flag_bisiesto = 1
      Fin_Si
    Fin_Si
  Fin_Si
  Si (flag_bisiesto == 1) Entonces
    Escribir(year, 'es bisiesto')
  Sino
    Escribir(year, 'no es bisiesto')
  Fin_Si
Fin
```



Ejemplo de repaso

Ejemplo – Año bisiesto:

Pseudocódigo

```
Inicio
    flag_bisiesto = 0
    Leer(year)
    Si(year%400 == 0) Entonces
        flag_bisiesto = 1
    Sino
        Si(year%100 != 0) Entonces
            Si(year%4 == 0) Entonces
                flag_bisiesto = 1
            Fin_Si
        Fin_Si
    Fin_Si
    Si (flag_bisiesto == 1) Entonces
        Escribir(year, 'es bisiesto')
    Sino
        Escribir(year, 'no es bisiesto')
    Fin_Si
Fin
```

Python

```
# Inicializacion
flag_bisiesto = 0

# Entradas
year = int(input("Digite el año: "))

# Proceso
if(year%400 == 0):
    flag_bisiesto = 1
else:
    if(year%100 == 0):
        if(year%4 == 0):
            flag_bisiesto = 1

# Salida
if (flag_bisiesto == 1):
    print(f"{year} es bisiesto")
else:
    print(f"{year} no es bisiesto")
```

[Link de ejecución](#)

Variable bandera (flag)

- **Una bandera (flag)** es una variable que almacena un estado (**0/1** o **Falso/Verdadero**) para recordar si una condición se cumplió o no durante la ejecución.
- Funciona como un interruptor de luz o una bandera física: abajo (0) indica estado normal y arriba (1) señala que "algo ocurrió".
- **Mecánica en el código:**
 - **Inicializar:** Nace con un valor por defecto (ej. **flag = 0** se asume que no es bisiesto).
 - **Cambiar:** Si el algoritmo detecta la excepción, la "levanta" (**flag = 1**).
 - **Consultar:** Al final, solo se evalúa la bandera para decidir qué mensaje mostrar.
- Si observa el código anterior, la ventaja de usar la bandera **flag_bisiesto**, separa la evaluación lógica de la impresión de resultados, evitando repetir instrucciones de salida en cada bloque del código.

Ejemplo de repaso

Ejemplo – Año bisiesto:



Datos de entrada:

- Año

Datos de salida:

- Decir si el año es bisiesto o no

Definición de variables:

- Entrada/salida:
 - **year:** Año.

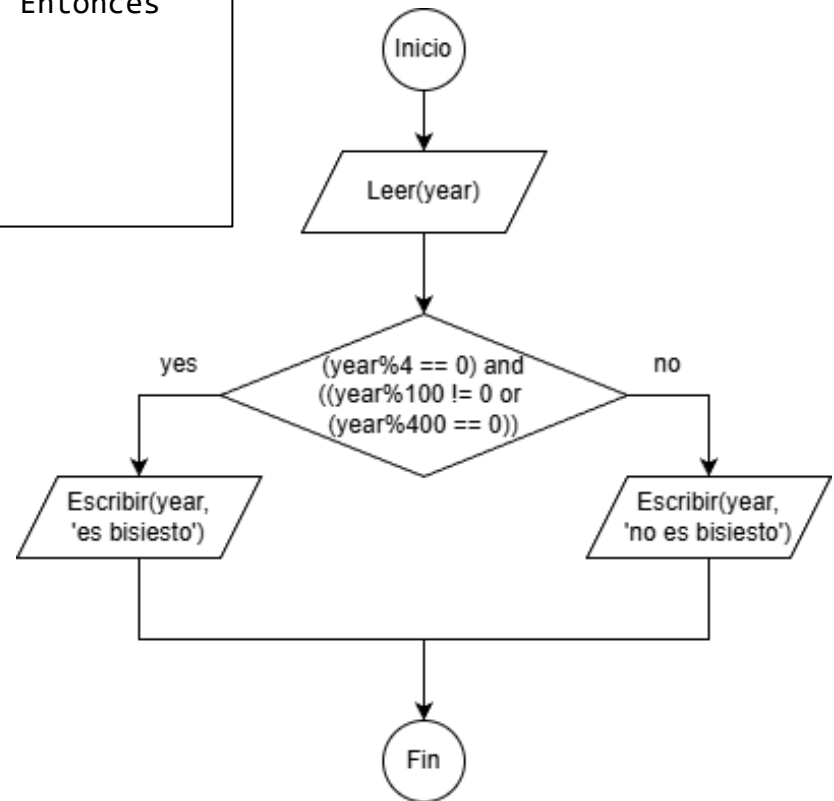
Alternativa doble

Ejemplo – Año bisiesto:

Pseudocódigo

```
Inicio
  Leer(year)
  Si((year%4 == 0)and(year%100 != 0 or year%400 == 0)) Entonces
    Escribir(year, 'es bisiesto')
  Sino
    Escribir(year, 'no es bisiesto')
  Fin_Si
Fin
```

Note que el uso de conectores lógicos permitió decir en una sola condición cuanto un año es bisiesto, lo cual ahorro el uso de condicionales anidados.



Alternativa doble

Pseudocódigo

```
Inicio
  Leer(year)
  Si((year%4 == 0)and(year%100 != 0 or year%400 == 0)) Entonces
    Escribir(year, 'es bisiesto')
  Sino
    Escribir(year, 'no es bisiesto')
  Fin_Si
Fin
```

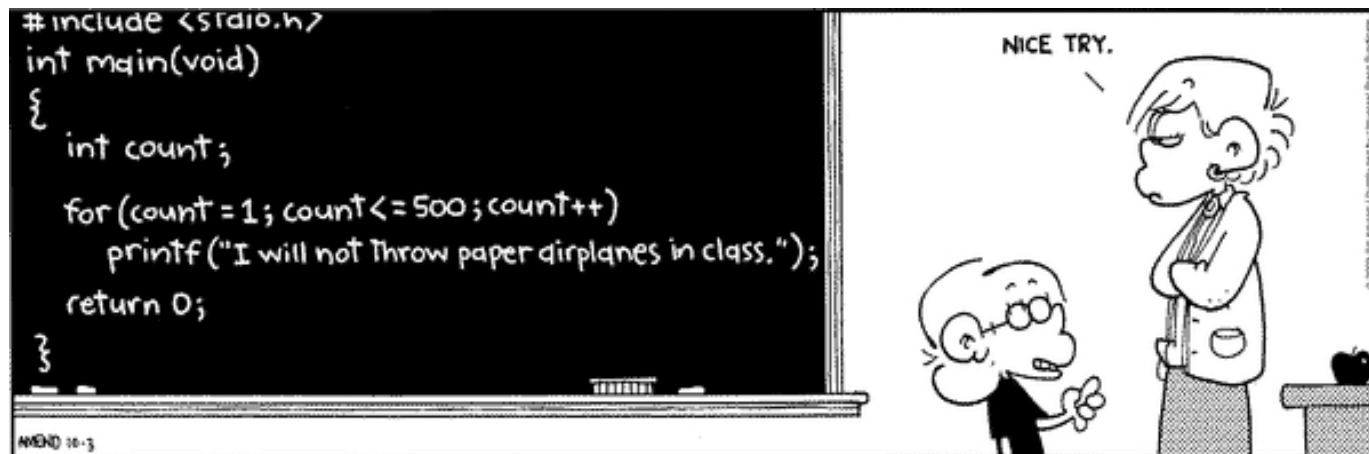
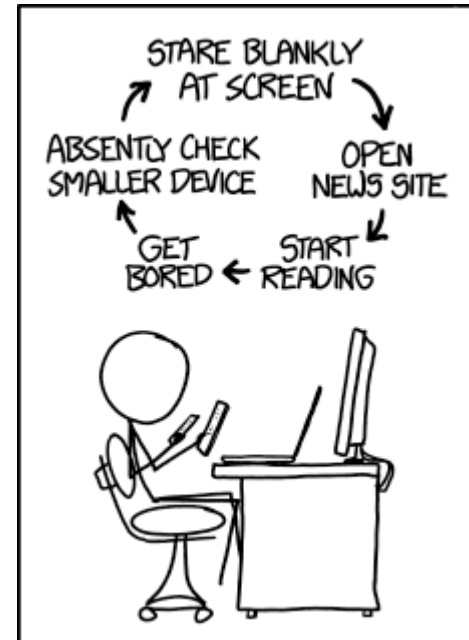
Python

```
# Entradas
year = int(input("Digite el año: "))

# Proceso y salida
if((year%4 == 0) and
   (year%100 != 0) or (year%400 == 0)):
    print(year, "es bisiesto")
else:
    print(year, "no es bisiesto")
```

[Link de ejecución](#)

Ciclos



Ciclos

Veamos como punto de partida los siguientes problemas:

- **Problema 1:** Hacer un programa que muestre los primeros 10 números en pantalla.

```
Inicio
  Escribir("1, 2, 3, 4, 5, 6, 7, 8, 9, 10")
Fin
```

- **Problema 2:** Hacer un programa que muestre los primeros 100 números en pantalla.

```
Inicio
  Escribir("1, 2, 3, 4, 5, 6, 7, 8, 9, 10,
          11, 12, 13, 14, 15, 16, 17, 18, 19, 20,
          21, 22, 23, 24, 25, 26, 27, 28, 29, 30,
          31, 32, 33, 34, 35, 36, 37, 38, 39, 40,
          41, 42, 43, 44, 45, 46, 47, 48, 49, 50,
          51, 52, 53, 54, 55, 56, 57, 58, 59, 60,
          61, 62, 63, 64, 65, 66, 67, 68, 69, 70,
          71, 72, 73, 74, 75, 76, 77, 78, 79, 80,
          81, 82, 83, 84, 85, 86, 87, 88, 89, 90,
          91, 92, 93, 94, 95, 96, 97, 98, 99, 100")
Fin
```

Ciclos

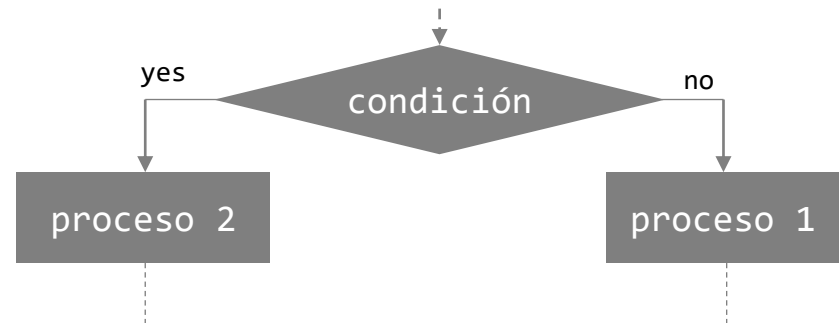
Veamos como punto de partida los siguientes problemas:

- **Problema 3:** Hacer un programa que muestre los primeros 1000 números en pantalla.

Read v1,v2,...

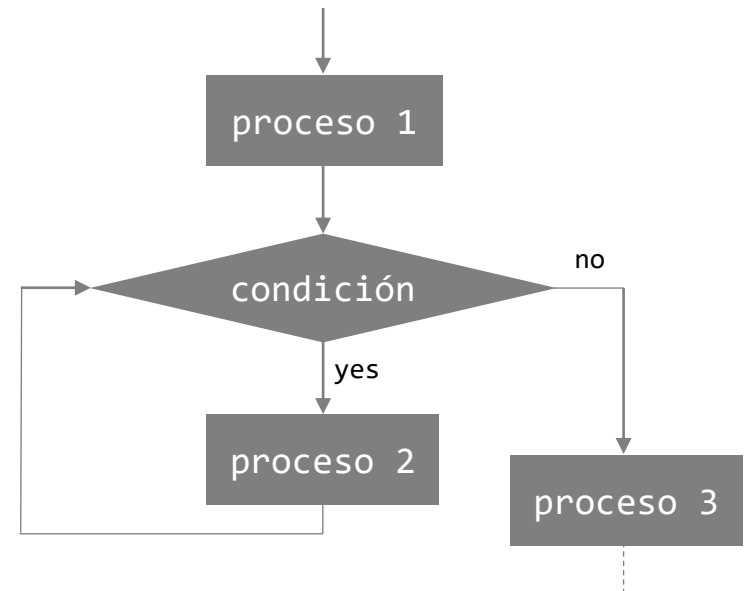
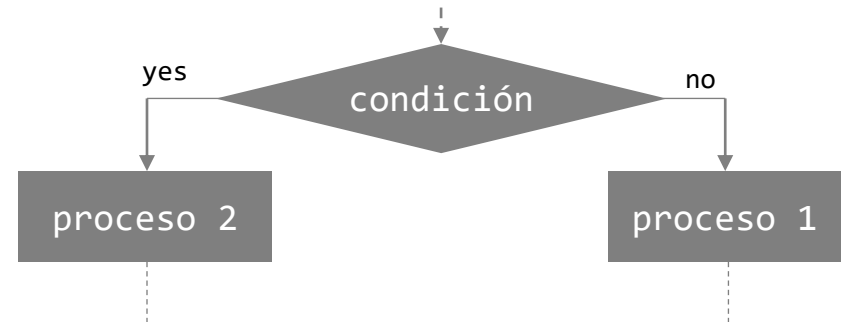
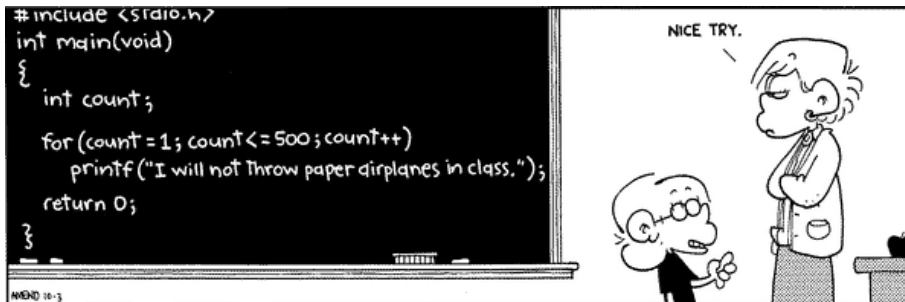
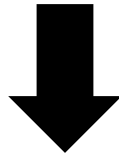
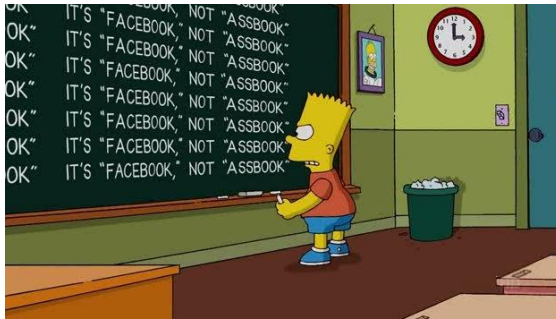
Print v1,v2,...

$c \leftarrow a + b$



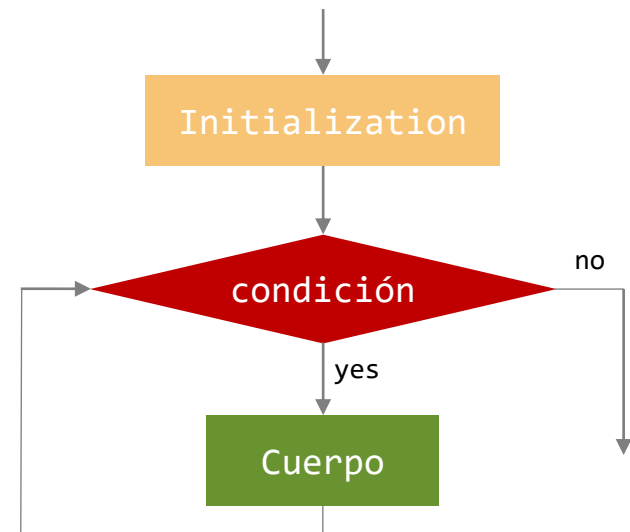
Ciclos

Conclusión: Necesitamos una nueva estructura, conocida como **ciclo**, para aquellas situaciones donde un conjunto de instrucciones se llevan a cabo de manera repetitiva.



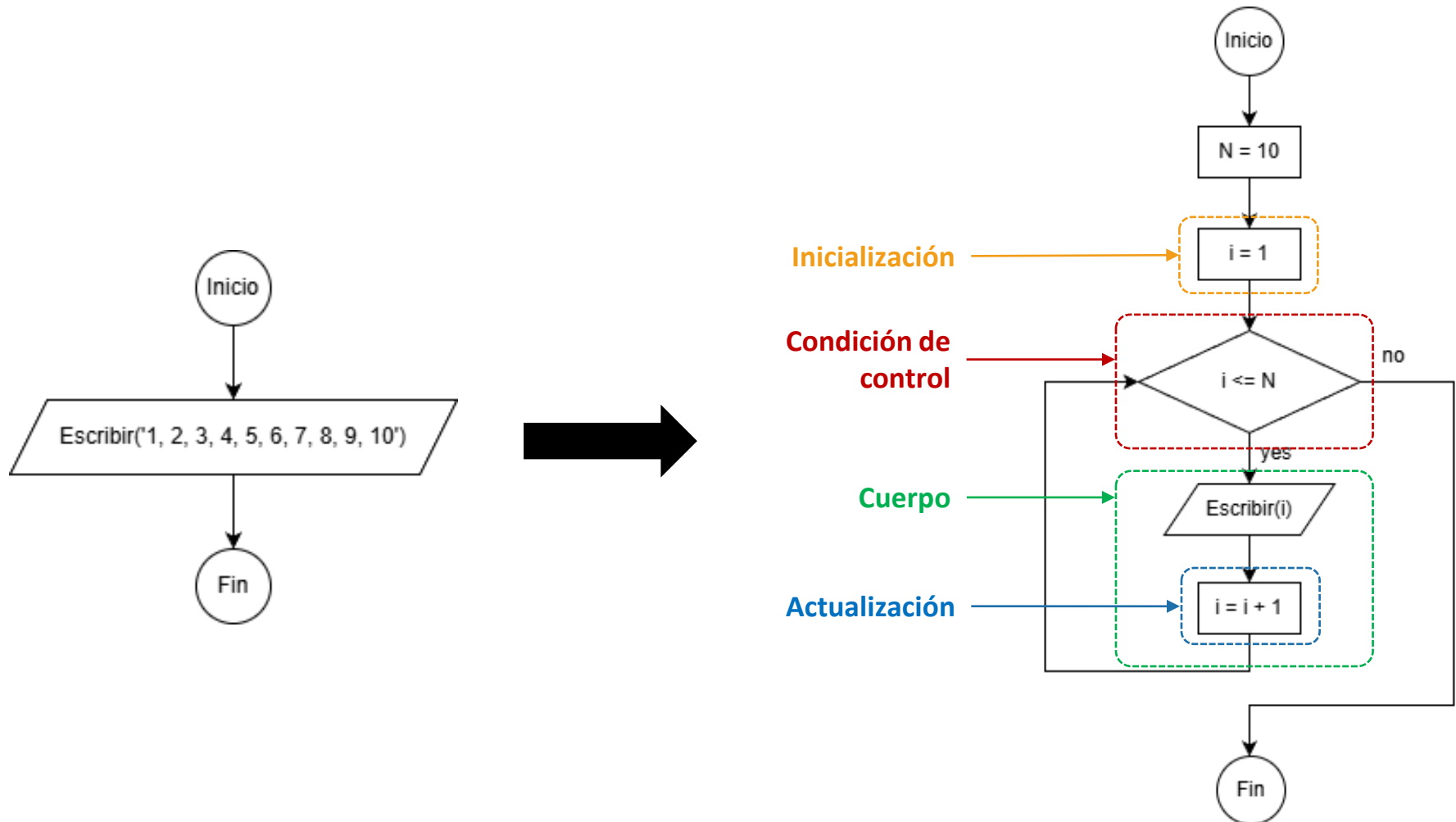
Ciclos

- Un **ciclo** (bucle o iteración) es una estructura que repite un conjunto de instrucciones mientras se cumpla una condición.
- Para describir un ciclo se requieren los siguientes componentes :
 - **Inicialización:** parte donde se establece el valor inicial de la variable de control, antes de que el ciclo arranque.
 - **Condición de control o parada:** Expresión lógica evaluada en cada iteración. Si es verdadera, el ciclo continúa; si es falsa, se detiene
 - **Cuerpo del ciclo:** Instrucciones que se ejecutan repetidamente. Incluye la modificación obligatoria de la variable de control para asegurar que el ciclo pueda terminar.



Ciclos - Ejemplo 1

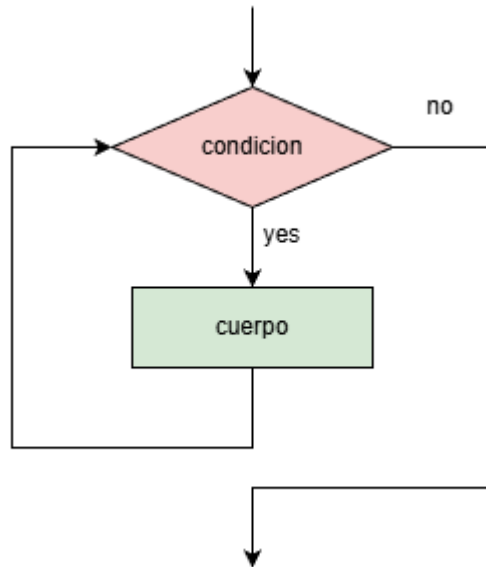
Retomando el problema inicial de imprimir los $N = 10$ primeros números, el uso de ciclos permite solucionar fácilmente la cantidad de números a imprimir cambiando solo el valor de N .



Ciclos

Representación en pseudocódigo

Diagrama de flujo



Pseudocódigo

```
Mientras (condicion) Haga  
    instrucciones cuerpo  
Fin_Mientras
```

Python

```
while (condicion):  
    instrucciones cuerpo
```

Observaciones importantes:

- Note que aunque la inicialización es importante para el ciclo, es previa a este (por eso no aparece).
- Dentro del cuerpo del ciclo están las instrucciones propias de lo que en este se debe realizar y la actualización de la variable de condición

Ciclos – Ejemplo 1

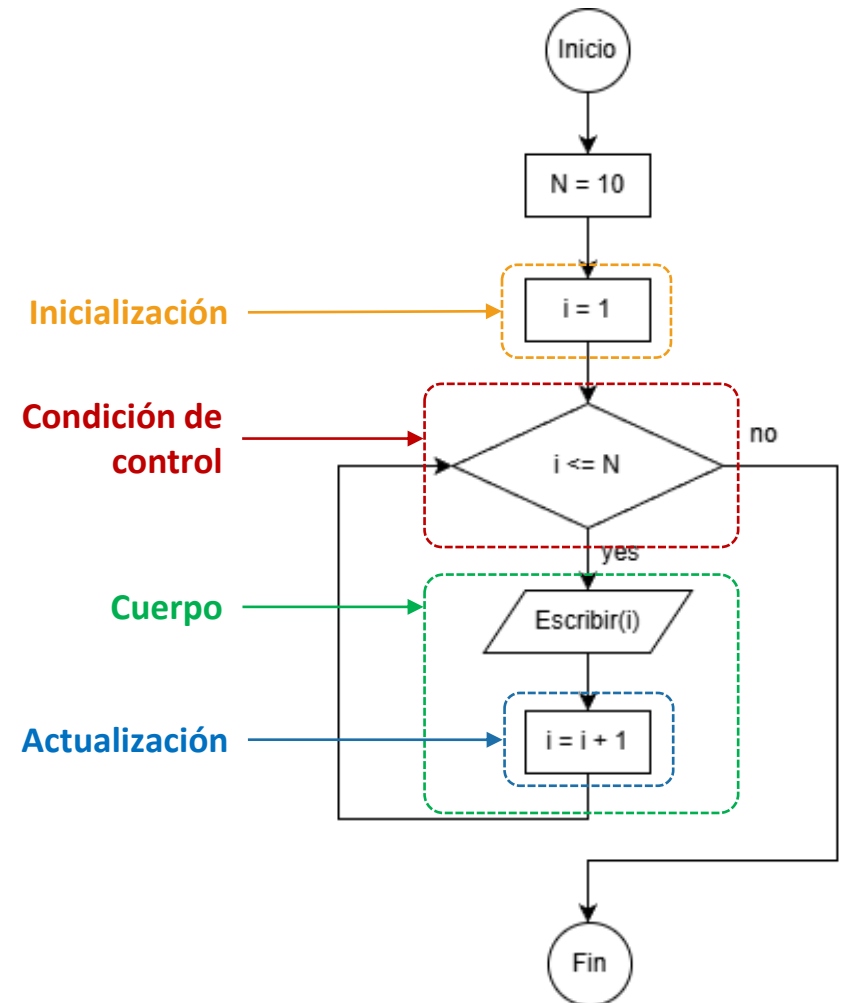
Pseudocódigo

```
Inicio
  N = 10
  i = 1
  Mientras (i <= N) Haga
    Escribir(i)
    i = i + 1
  Fin_Mientras
Fin
```

Python

```
N = 10
i = 1
while i <= N:
    print(i)
    i = i + 1
```

[Link de ejecución](#)



Ciclos - Ejemplo 2

Escriba un algoritmo que calcule la suma de los primeros **N** números enteros mayores que 0, donde **N** es un dato de entrada.



Datos de entrada:

- Cantidad de números

Datos de salida:

- Suma de los números

Definición de variables:

• Entrada/salida:

- **N**: Cantidad de números a sumar
- **suma**: Suma de los números generados (*acumulador*).

• Variables auxiliares:

- **num**: Numero entero generado (*contador*)

Proceso:

- Inicializar la suma en 0.
- Inicializar el número a generar (num) en 1.
- Repetir mientras num sea menor o igual que N:
 - Acumular num en suma.
 - Incrementar num en 1.
- Imprimir el resultado de la suma.

Ciclos – Ejemplo 2

Escriba un algoritmo que calcule la suma de los primeros **N** números enteros mayores que 0, donde **N** es un dato de entrada.

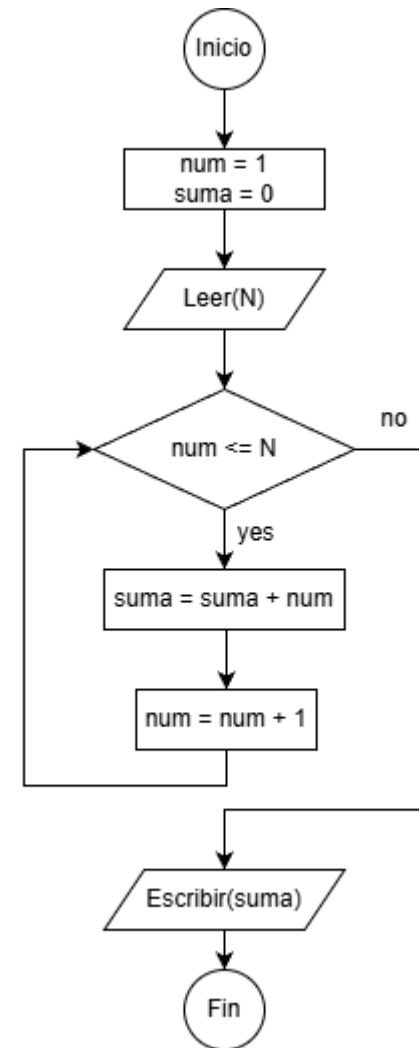
Pseudocódigo

```
Inicio
  num = 1
  suma = 0
  Leer(N)
  Mientras (num <= N) Haga
    suma = suma + num
    num = num + 1
  Fin_Mientras
  Escribir(suma)
Fin
```

Python

```
num = 1
suma = 0
N = int(input('Ingrese la cantidad de números a sumar: '))
while num <= N:
    suma = suma + num
    num = num + 1
print(f"La suma de los {N} primeros números da {suma}")
```

[Link de ejecución](#)



Ciclos – Ejemplo 2

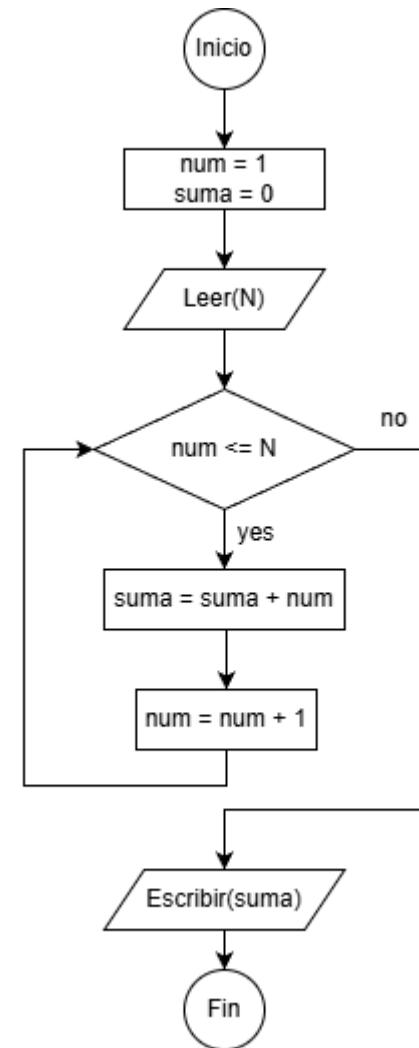
Escriba un algoritmo que calcule la suma de los primeros **N** números enteros mayores que 0, donde **N** es un dato de entrada.

```
Inicio
  num = 1
  suma = 0
  Leer(N)
  Mientras (num <= N) Haga
    suma = suma + num
    num = num + 1
  Fin_Mientras
  Escribir(suma)
Fin
```

Prueba de escritorio

N	suma
0	0
4	10

N	num	suma



Ciclos – Ejemplo 2

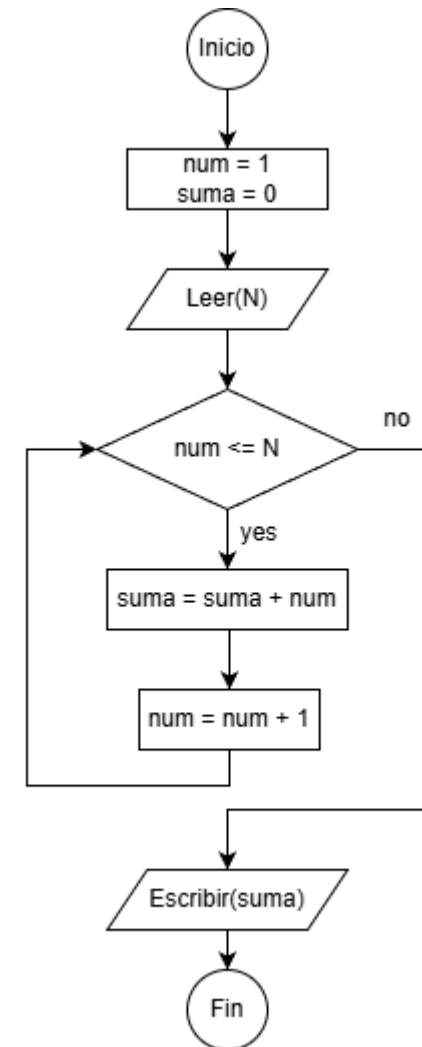
Escriba un algoritmo que calcule la suma de los primeros **N** números enteros mayores que 0, donde **N** es un dato de entrada.

```
Inicio
  num = 1
  suma = 0
  Leer(N)
  Mientras (num <= N) Haga
    suma = suma + num
    num = num + 1
  Fin_Mientras
  Escribir(suma)
Fin
```

Prueba de escritorio (solución)

N	suma
4	10

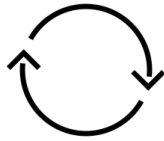
N	num	suma
?	?	?
4	1	0
	2	1
	3	3
	4	6
	5	10



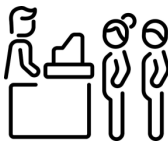
Tipos de variables útiles en ciclos

Todo ciclo suele apoyarse en algunos tipos de variables que cumplen roles específicos:

- **Variable de control**



- **Contador**



- **Acumulador**



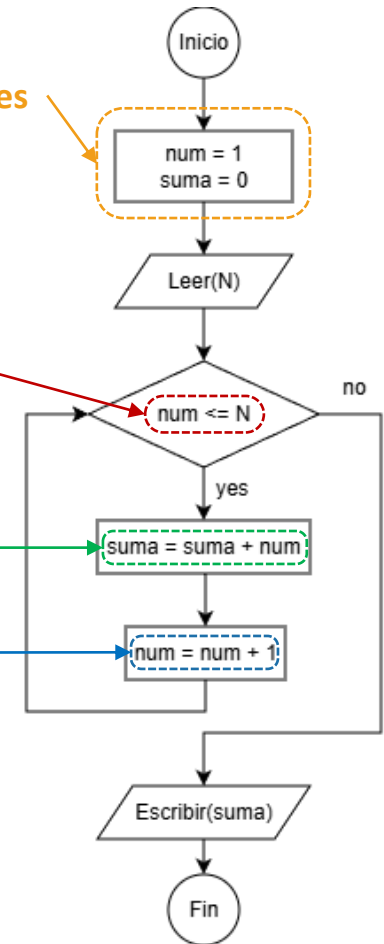
- **Bandera**
- **Centinela**

Inicialización de variables

Variable de control

Acumulador

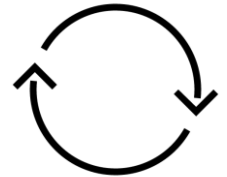
Contador



Variable de control

- Como previamente se menciono, es la variable que se evalúa en la condición del ciclo para decidir si continúa o se detiene.
- Debe inicializarse antes de entrar al ciclo y modificarse dentro del cuerpo, de forma que eventualmente la condición se vuelva falsa.
- En el ejemplo previo (ver pseudocodigo), **num** es la variable de control: aparece en la condición **num <= N** y se actualiza con **num = num + 1**.

```
Inicio
  num = 1                                # Inicializacion
  suma = 0
  Leer(N)
  Mientras (num <= N) Haga                # Comparación
    suma = suma + num
    num = num + 1                         # Actualización
  Fin_Mientras
  Escribir(suma)
Fin
```



Nota: la variable de control frecuentemente toma la forma de un contador (cuando avanza en una cantidad fija), pero también puede ser una bandera booleana o depender de un centinela, según el tipo de ciclo.

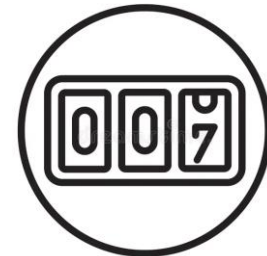
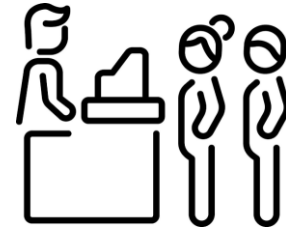
Contador

- Un contador es variable numérica que sirve para contar cuántas veces se repite una acción o se cumple una condición.
- Debe inicializarse a un valor conocido antes de usarse.
- Su característica principal es que su valor se incrementa o decrementa sistemáticamente en una **cantidad constante**.

contador = contador + CONSTANTE

- **Ejemplos:**

- `num = num + 1` (Ejemplo anterior)
- `estudiantes = estudiantes + 1`
- `i = i + 1`
- `j = j - 1`
- `tiempo_espera = tiempo_espera - 1`



Acumulador

- Un acumulador es variable que sirve para acumular (o totalizar) el resultado de una operación que se repite.
- Debe inicializarse a un valor conocido antes de usarse. Generalmente en 0 para sumas o 1 para productos.
- Se modifica en un valor **variable** (no constante) en cada iteración.

contador = **contador** (operación) **variable**

- **Ejemplos:**

- `suma = suma + num` (Ejemplo anterior)
- `ahorro = ahorro + dinero`
- `total_venta = total_venta + factura`
- `factorial = factorial*i`
- `j = j - 1`
- `saldo_favor = saldo_favor - gasto`



Ciclos - Ejemplo 3

Haga un programa que calcule el factorial de un número entero no negativo n .

Recuerde cuando $n = 0$, $0! = 1$, y para cualquier otro valor el factorial esta dado por:

$$n! = 1 \times 2 \times 3 \times \cdots \times (n - 2) \times (n - 1) \times n$$

Ciclos - Ejemplo 3



Datos de entrada:

- Numero al que se le calculara el factorial

Datos de salida:

- Factorial

Definición de variables:

Entrada/salida:

- **n**: Numero al que de le calculara el factorial
- **fact**: factorial del numero (*acumulador*).
- **Variables auxiliares:**
 - **i**: Contador que recorre los valores desde 1 hasta n (para generar el i-esimo numero)

Proceso:

- Inicializar variables ($i = 1$, $fac = 1$)
- Solicitar el numero
- Repetir mientras **i** sea menor o igual que **n**:
 - Calcular el factorial i-esimo.
 - Incrementar **i** en 1.
- Imprimir el resultado del factorial.

Ciclos – Ejemplo 3

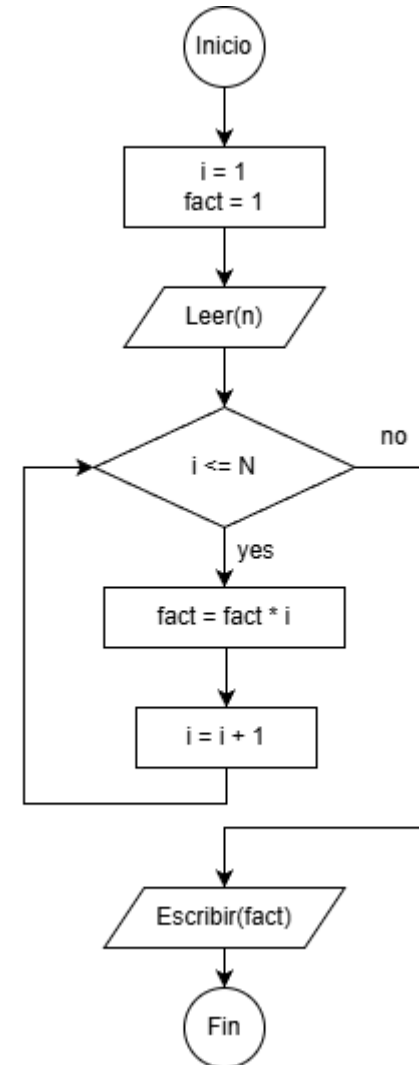
Pseudocódigo

```
Inicio
  i = 1
  fact = 1
  Leer(n)
  Mientras (i <= n) Haga
    fact = fact * i
    i = i + 1
  Fin_Mientras
  Escribir(n, '!= ', fact)
Fin
```

Python

```
i = 1
fact = 1
n = int(input('Digite el numero (>= 0): '))
while i <= n:
    fact = fact * i
    i = i + 1
print(f"{n}!= {fact}")
```

[Link de ejecución](#)



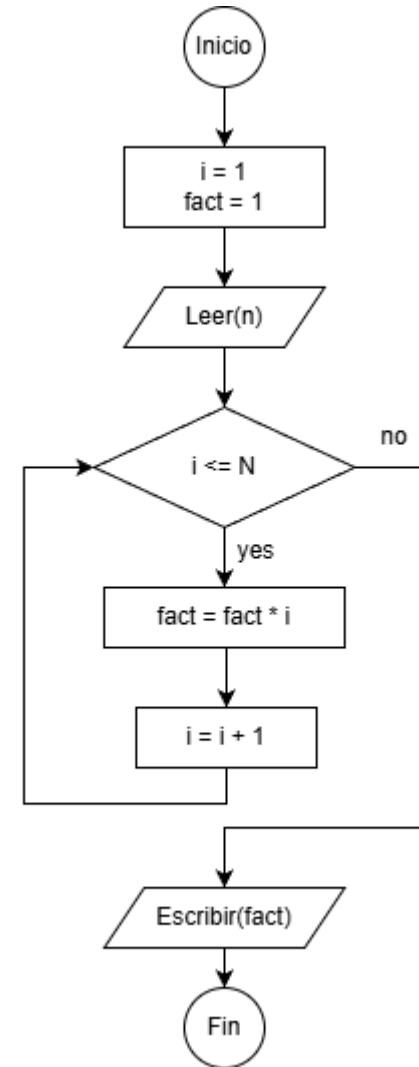
Ciclos – Ejemplo 3

```
Inicio
  i = 1
  fact = 1
  Leer(n)
  Mientras (i <= n) Haga
    fact = fact * i
    i = i + 1
  Fin_Mientras
  Escribir(n, '!= ', fact)
Fin
```

Prueba de escritorio

n	factorial
0	1
3	6
5	120

n	i	fact



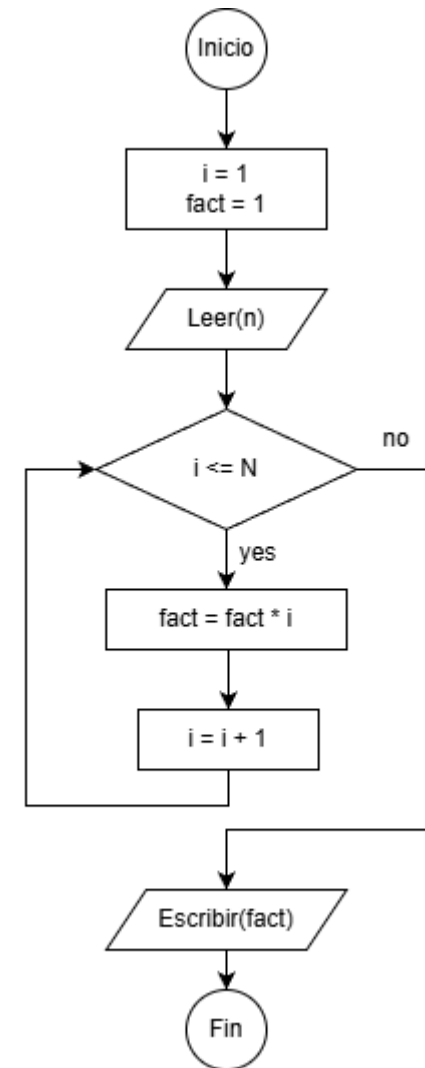
Ciclos – Ejemplo 3

```
Inicio
  i = 1
  fact = 1
  Leer(n)
  Mientras (i <= n) Haga
    fact = fact * i
    i = i + 1
  Fin_Mientras
  Escribir(n, '! = ', fact)
Fin
```

Prueba de escritorio (solución)

n	factorial
5	120

N	i	fact
?	?	?
5	1	1
	2	2
	3	6
	4	24
	5	120
	6	



Ciclos - Ejemplo 4

Imprimir los números del n a 1 alternando el signo, por ejemplo: 6, -5, 4, -3, 2, -1.

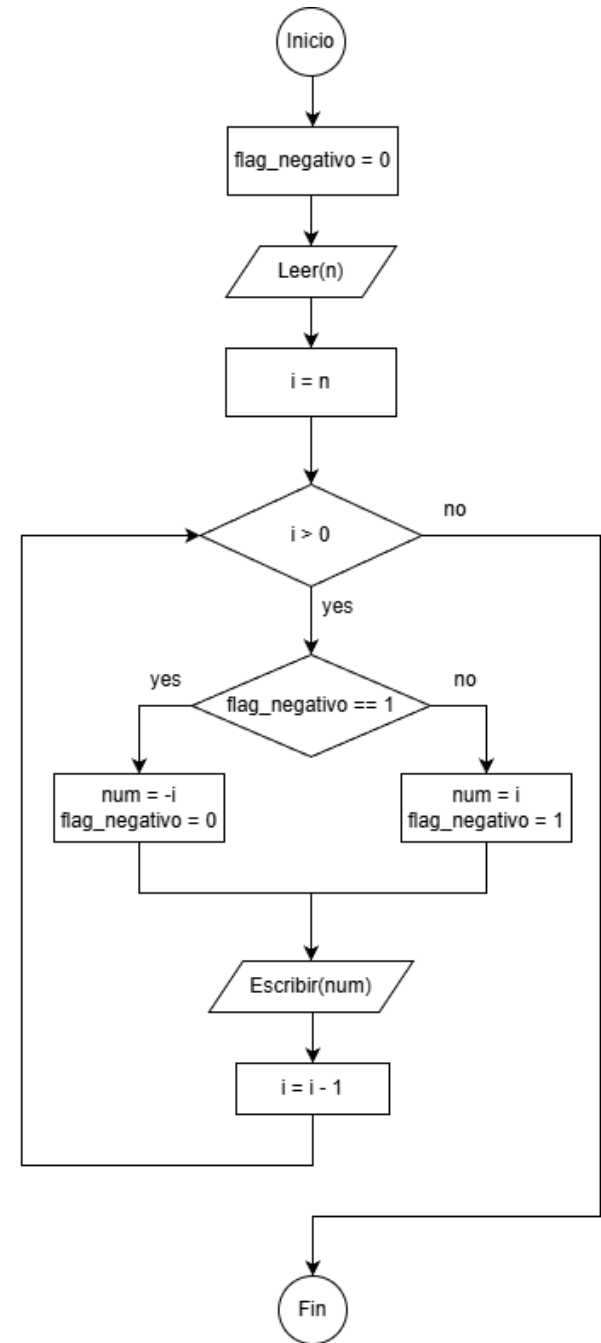


Variables auxiliares

flag_negativo
i

```
Inicio
  flag_negativo = 0
  Leer(n)
  i = n
  Mientras (i > 0) Haga
    Si flag_negativo == 1 Entonces
      num = -i
      flag_negativo = 0
    Sino
      num = i
      flag_negativo = 1
    Fin_Si
    Escribir(num)
    i = i - 1
  Fin_Mientras
Fin
```

[Link de ejecución](#)



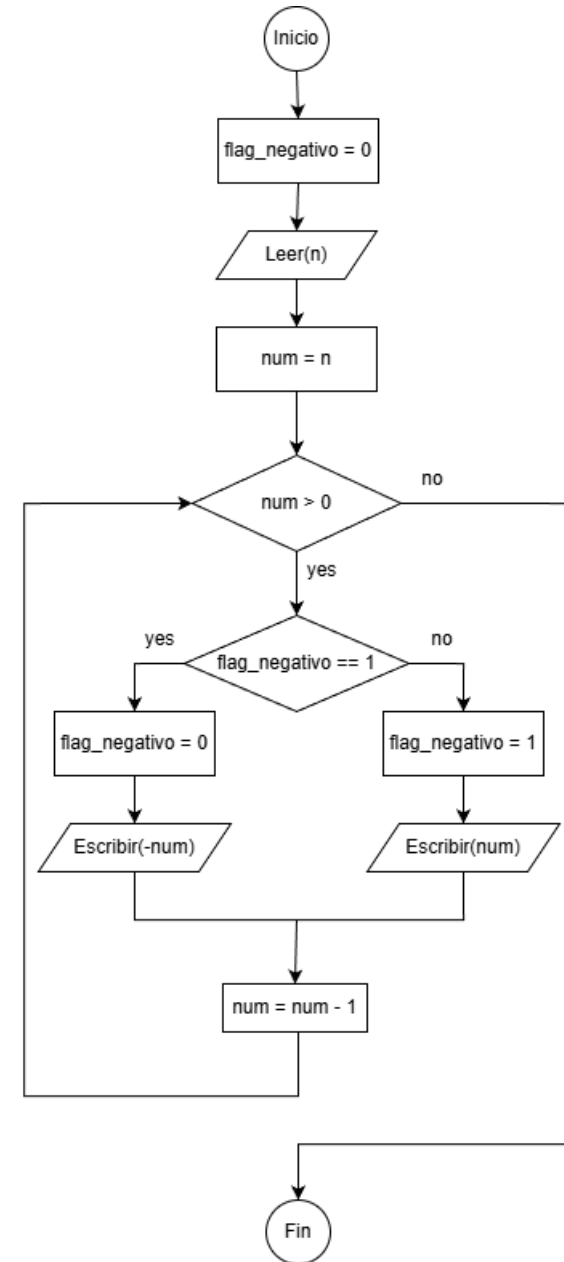
Ciclos - Ejemplo 4

Imprimir los números del n a 1 alternando el signo, por ejemplo: 6, -5, 4, -3, 2, -1.



```
Inicio
  flag_negativo = 0
  Leer(n)
  num = n
  Mientras (num > 0) Haga
    Si flag_negativo == 1 Entonces
      flag_negativo = 0
      Escribir(-num)
    Sino
      flag_negativo = 1
      Escribir(num)
    Fin_Si
    num = num - 1
  Fin_Mientras
Fin
```

[Link de ejecución](#)



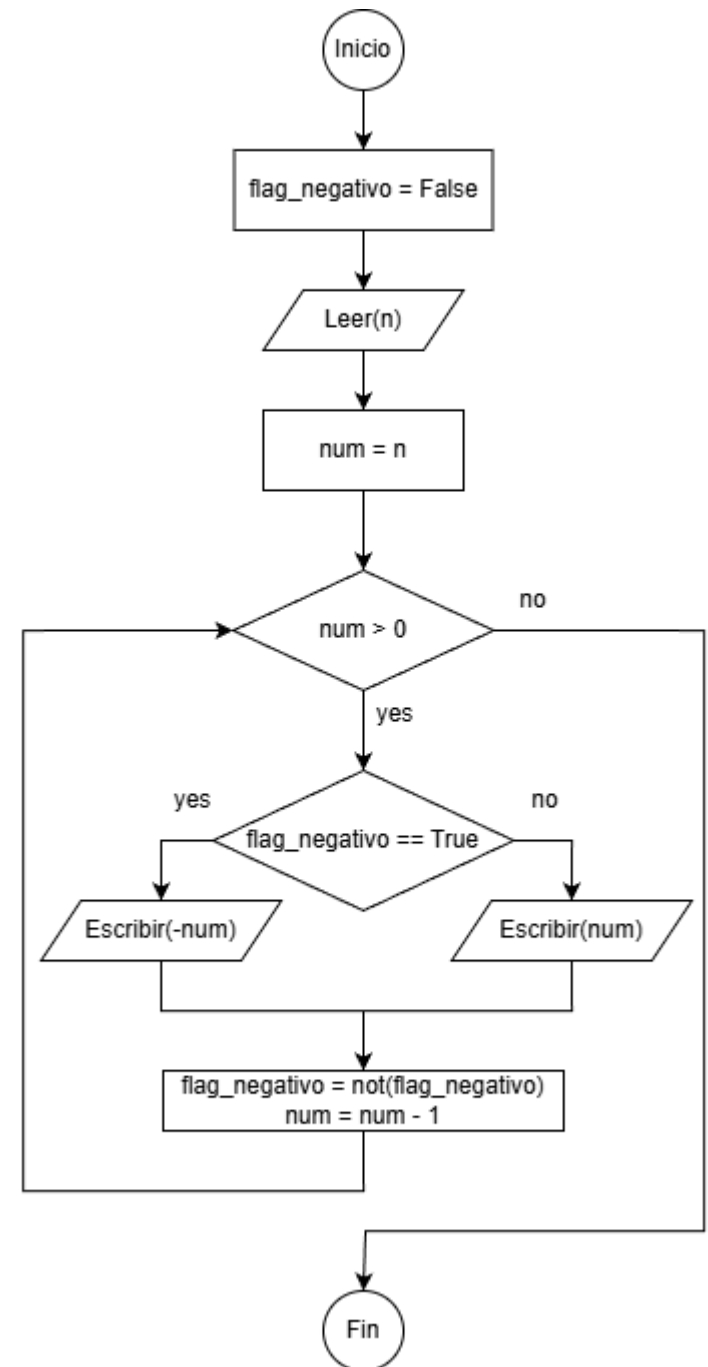
Ciclos - Ejemplo 4

Imprimir los números del n a 1 alternando el signo, por ejemplo: 6, -5, 4, -3, 2, -1.



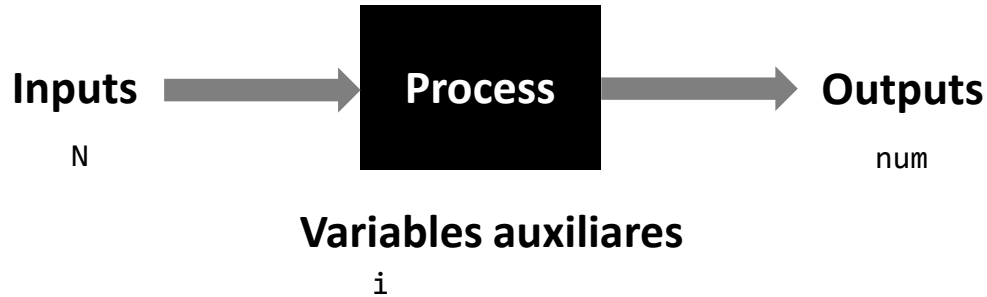
```
Inicio
  flag_negativo = False
  Leer(n)
  num = n
  Mientras (num > 0) Haga
    Si flag_negativo == True Entonces
      Escribir(-num)
    Sino
      Escribir(num)
    Fin_Si
    flag_negativo = not(flag_negativo)
    num = num - 1
  Fin_Mientras
Fin
```

[Link de ejecución](#)



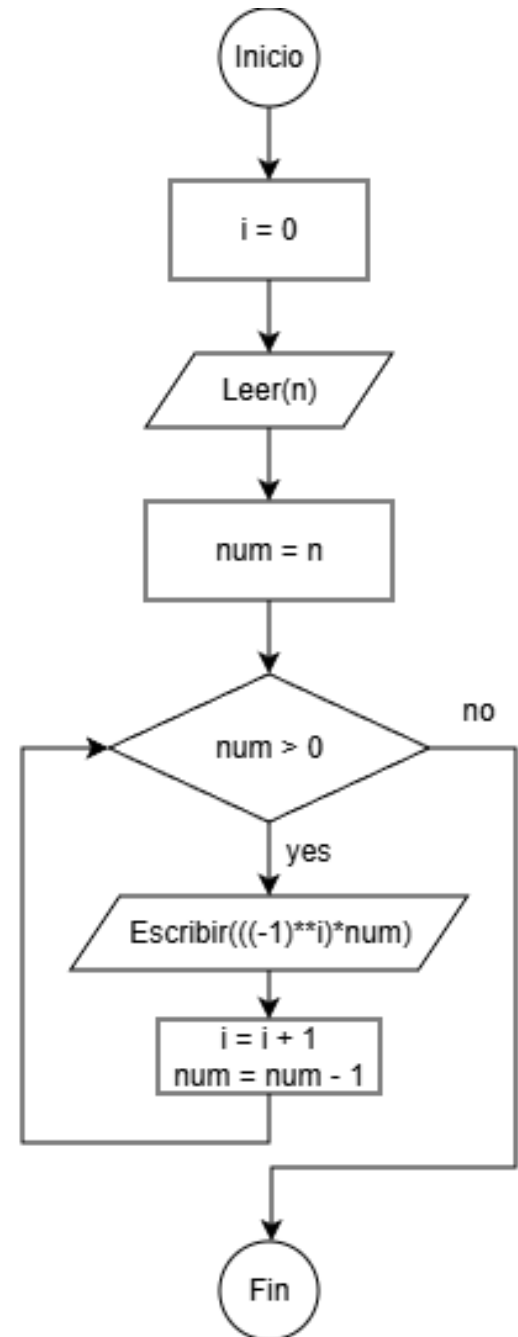
Ciclos - Ejemplo 4

Imprimir los números del n a 1 alternando el signo, por ejemplo: 6, -5, 4, -3, 2, -1.



```
Inicio
i = 0
Leer(n)
num = n
Mientras (num > 0) Haga
    Escribir(((−1)**i)*num)
    i = i + 1
    num = num - 1
Fin_Mientras
Fin
```

[Link de ejecución](#)



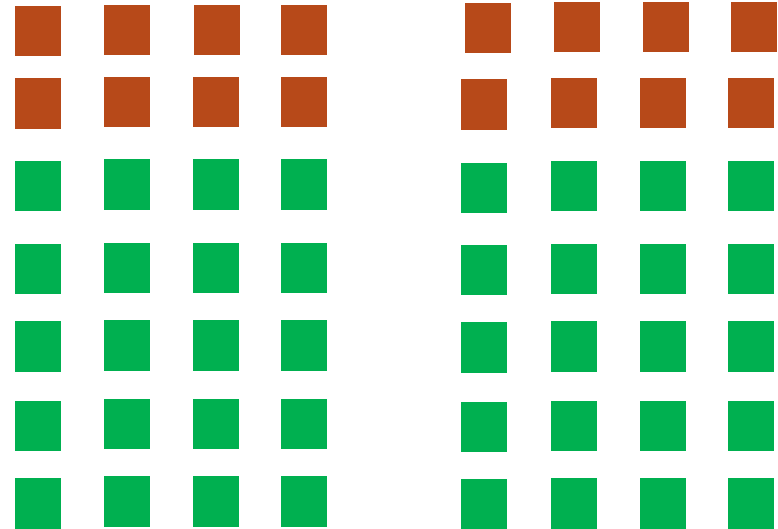
Ciclos – Ejemplo 5

El señor Burns, como buen capitalista que es, decidió construir un cine en Springfield. El cine es pequeño: tiene capacidad para 8 puestos VIP (a \$7000 cada uno) y 40 puestos normales (a \$5000 cada uno).

Para cada función, el señor Burns no sabe de antemano cuántos clientes van a asistir — puede que se llenen todos los puestos, o que la venta se cierre antes con puestos disponibles. Desarrolle una aplicación que:

- Registre la venta de una boleta solicitando al cajero el tipo de puesto (VIP o normal), y repita este registro hasta que ocurra una de estas dos condiciones: se agoten los 48 puestos, o el cajero ingrese -1 como tipo de puesto para indicar que no hay más clientes y cerrar la venta.
- Si el tipo de puesto solicitado ya alcanzó su capacidad máxima, no contabilizar esa venta, informarlo al cajero, y volver a solicitar el tipo de puesto (sin que esto se interprete como el cierre de la venta)
- Obtener, para cada tipo de puesto, el total de asistentes y la ganancia obtenida.
- Cuando se cierre la venta de boletas para la función (por cualquiera de las dos condiciones anteriores), imprimir las estadísticas: totales y porcentajes de asistentes y de dinero recaudado, por tipo de entrada.

Ciclos – Ejemplo 5



VIP: \$7000

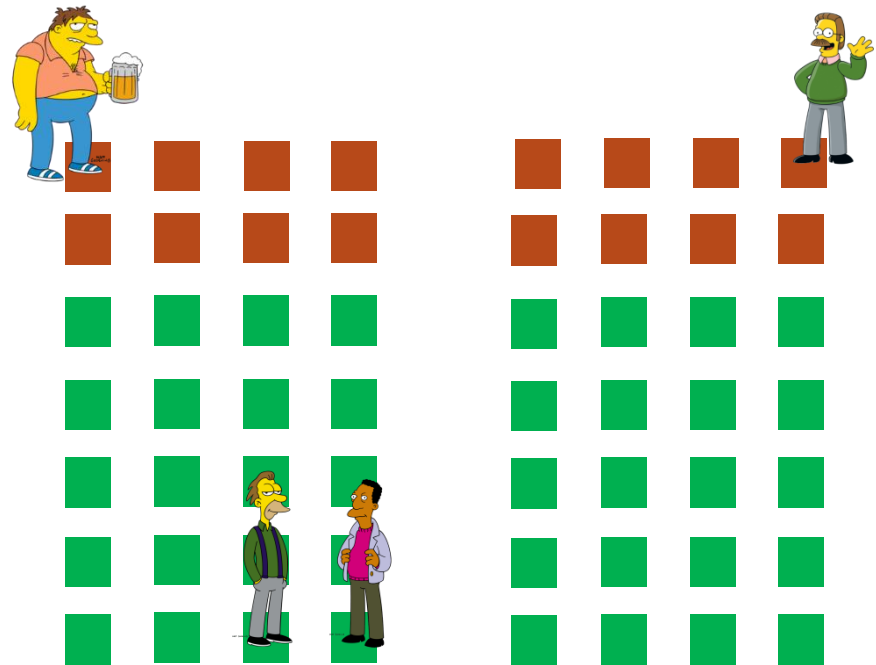


Normal: \$5000

Ciclos – Ejemplo 5



Puestos VIP	2
Puestos Normal	2
Total VIP	14000
Total Normal	10000
Asistentes	4
Total	24000



VIP: \$7000



Normal: \$5000

Ciclos

- El **Ejemplo 5** es distinto a los ejemplos **1** a **4**: en aquellos, el número de repeticiones del ciclo era un dato conocido antes de empezar (N). En el caso del **ejemplo 5**, el vendedor de boletas del cine, no sabe de antemano cuántos clientes van a llegar a la función, así que tampoco se conoce con anticipación cuántas boletas se van a vender.
- Cuando el número de iteraciones no se conoce de antemano, no basta con contar hasta un valor fijo. Se necesita otra forma de decidir cuándo termina el ciclo. Para esto existen dos herramientas: las **banderas (flags)** y los **centinelas**.



Tipos de problemas con ciclos



Problema

Numero conocido de iteraciones

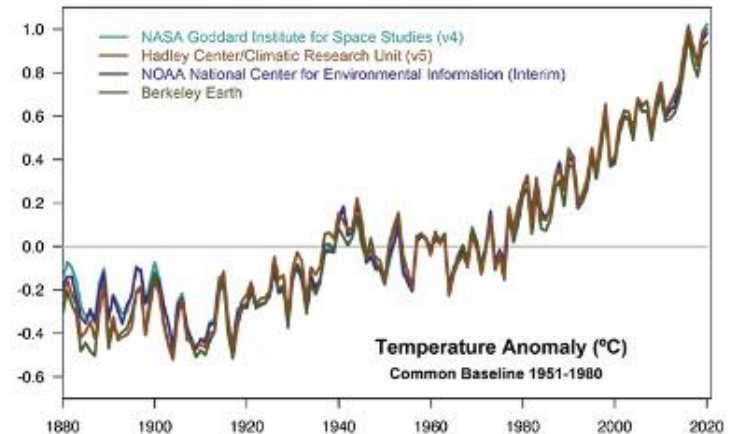
Numero desconocido de iteraciones

Tipos de problemas con ciclos

Los problemas con ciclos se pueden clasificar en dos tipos:

1. Problemas con iteraciones conocidas: En este caso, el numero de iteraciones es conocida.

- Promedio de notas en un curso de Lógica y programación.
- Promedio de temperatura de una cantidad conocida de tomas.



Tipos de problemas con ciclos

Los problemas con ciclos se pueden clasificar en dos tipos:

2. **Problemas con numero de iteraciones desconocidas:** En estos problemas (como se menciona anteriormente), no se conocen el numero de iteraciones que el ciclo llevara a cabo.
- Asistentes a una sala de cine para ver la función de las 10:00 PM.
 - Carros vendidos por Gil Gunderson en una jornada laboral.



Ciclos – Ejemplo 6

Hacer un programa que lea una cantidad desconocida de notas (valores decimales entre 0.0 y 5.0) de los estudiantes de un curso, una a una.

Considere que la nota mínima aprobatoria es 3.0: una nota es **aprobada** si es mayor o igual a 3.0, y **reprobada** si es menor a 3.0.

Al finalizar la lectura, el programa deberá entregar:

- El porcentaje de estudiantes aprobados.
- El porcentaje de estudiantes reprobados.
- El promedio de las notas aprobadas.
- El promedio de las notas reprobadas.
- El promedio de las notas del curso.

Ciclos – Ejemplo 6



Datos de entrada:

- **Bandera** que indica si hay estudiante
- **Nota** del estudiante

Datos de salida:

- **Porcentaje** de estudiantes que ganaron
- **Porcentaje** de estudiantes que perdieron
- **Promedio** de notas de los que ganaron
- **Promedio** de notas de los que perdieron
- **Promedio** de notas del curso.

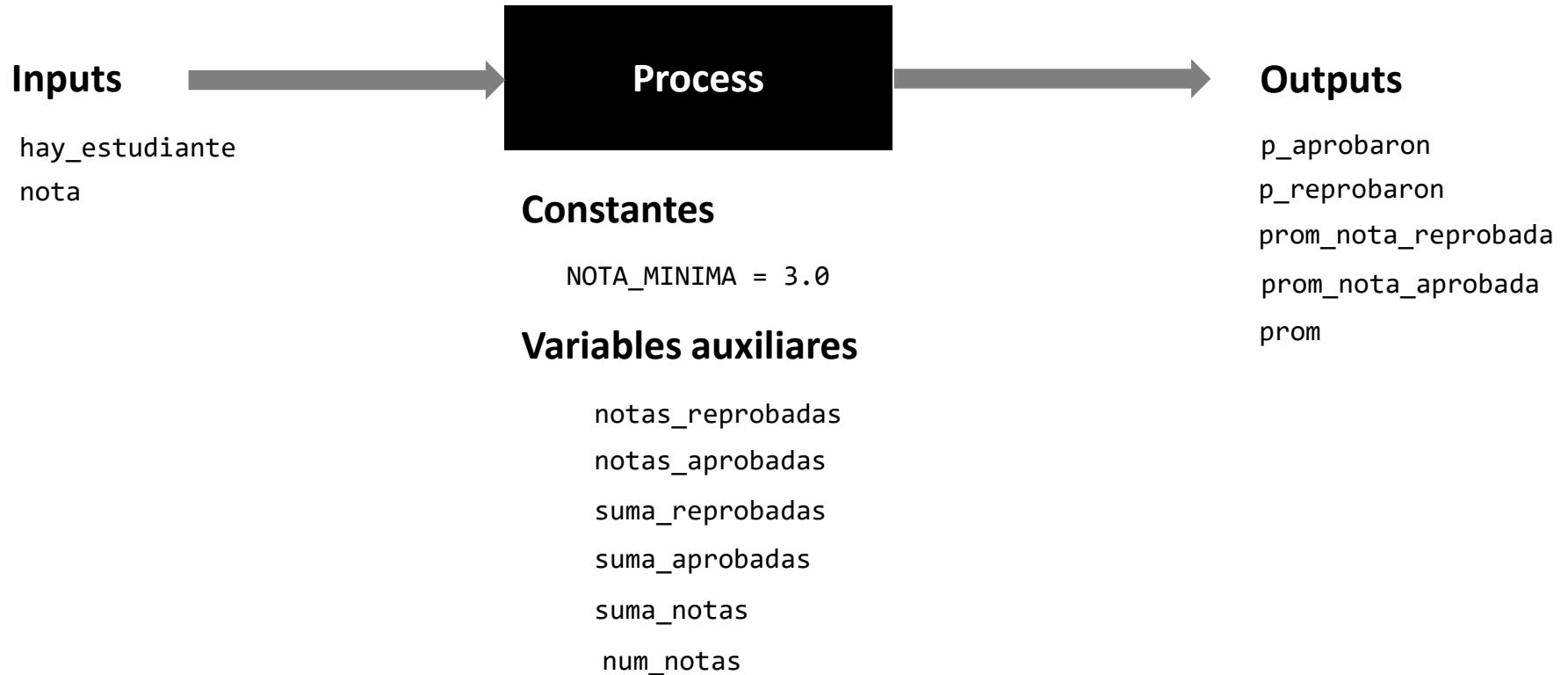
Definición de variables:

- **Entrada/salida:**
 - **hay_estudiante:** Bandera que indica si hay un estudiante por registrar
 - **nota:** Nota del estudiante.
 - **p_ganadores:** Porcentaje ganadores.
 - **p_perdedores:** Porcentaje perdedores.
 - **prom_nota_reprobada:** Promedio notas reprobadas.
 - **prom_nota_aprobada:** Promedio de notas aprobadas.
 - **prom:** Nota del estudiante

Definición de variables:

- **Variables auxiliares:**
 - **notas_reprobadas:** Numero de notas reprobadas (contador)
 - **suma_reprobadas:** Suma de notas reprobadas (acumulador).
 - **notas_aprobadas:** Numero de notas aprobadas (contador)
 - **suma_aprobadas:** Suma de notas reprobadas (acumulador).
 - **num_notas:** Numero de notas registradas.
 - **suma_notas:** Suma de las notas registradas.

Ciclos – Ejemplo 6



Ciclos – Ejemplo 6 - Bandera

```
Inicio
  NOTA_MINIMA = 3.0
  notas_reprobadas = 0
  notas_aprobadas = 0
  suma_reprobadas = 0
  suma_aprobadas = 0
  Leer(hay_estudiante)
  Mientras (hay_estudiante == 's') Haga
    Leer(nota)
    Si nota >= NOTA_MINIMA Entonces
      notas_aprobadas = notas_aprobadas + 1
      suma_aprobadas = suma_aprobadas + nota
    Sino
      notas_reprobadas = notas_reprobadas + 1
      suma_reprobadas = suma_reprobadas + nota
    Fin_Si
    Leer(hay_estudiante)
  Fin_Mientras
  num_notas = notas_reprobadas + notas_aprobadas
  suma_notas = suma_reprobadas + suma_aprobadas
  Si num_notas == 0 Entonces
    Escribir("No hay estudiantes")
  Sino
    p_aprobaron = (notas_aprobadas/num_notas)*100
    p_reprobaron = (notas_reprobadas/num_notas)*100
    prom_nota_reprobada = suma_reprobadas/notas_reprobadas
    prom_nota_aprobada = suma_aprobadas/notas_aprobadas
    prom = suma_notas/num_notas
    Escribir(p_aprobaron, p_reprobaron)
    Escribir(prom_nota_reprobada, prom_nota_aprobada)
    Escribir(num_notas)
  Fin_Si
Fin
```

Resultados esperados

Notas	4.0, 2.5, 3.0, 1.5, 4.8
Aprobaron	(3) 60%
Reprobaron	(2) 40%
Promedio nota aprobada	3.93
Promedio nota reprobada	2.0
Promedio general	3.16

Prueba de escritorio

Realizar la prueba de escritorio para verificar que el algoritmo este bien planteado

Nota: Este programa así como esta planeado tiene aun un bug grave. Encuéntrelo y corrijalo.

Ciclos – Ejemplo 6 - Bandera

Resultados esperados

Notas	4.0, 2.5, 3.0, 1.5, 4.8
Aprobaron	(3) 60%
Reprobaron	(2) 40%
Promedio nota aprobada	3.93
Promedio nota reprobada	2.0
Promedio general	3.16

Forma

```
...  
Leer(hay_estudiantes)  
Mientras (hay_estudiantes == 's') Entonces  
    Leer(nota)  
    ...  
    Leer(hay_est)  
Fin_Mientras  
...
```

Resultado de ejecución

```
¿Hay estudiante? (s/n): s  
Ingrese la nota del estudiante: 4  
¿Hay estudiante? (s/n): s  
Ingrese la nota del estudiante: 2.5  
¿Hay estudiante? (s/n): s  
Ingrese la nota del estudiante: 3.0  
¿Hay estudiante? (s/n): s  
Ingrese la nota del estudiante: 1.5  
¿Hay estudiante? (s/n): s  
Ingrese la nota del estudiante: 4.8  
¿Hay estudiante? (s/n): n  
Porcentaje de estudiantes que aprobaron: 60.0%  
Porcentaje de estudiantes que reprobaron: 40.0%  
Promedio de notas reprobadas: 2.0  
Promedio de notas aprobadas: 3.9333333333333336  
Promedio general de notas: 3.16
```

Bandera (flag)

- Es una variable que generalmente toma dos valores mutuamente excluyentes (por ejemplo, sí/no, encontrado/no encontrado).
- Al igual que el contador y el acumulador, debe inicializarse a un valor conocido antes de entrar al ciclo.
- Se usa de dos formas dentro de un ciclo:
 - **Como condición del ciclo**, para decidir si sigue iterando o se detiene (La variable `hay_estudiante` del ejemplo 6 repite el ciclo mientras `hay_estudiante == 's'`).
 - **Como estado interno**, para alternar o recordar una decisión dentro del cuerpo del ciclo, sin que necesariamente controle cuándo termina (variable `flag_negativo` del ejemplo 4 para alternar el signo en el ejemplo de la secuencia).
- La siguiente tabla muestra valores típicos que puede tomar una bandera:

Tipo de dato	Posibles valores
Lógico	True y False
Texto	'S' y 'N', 'Sí' y 'No', 'Encontrado' y 'No encontrado'
Numérico	0 y 1

Ciclos – Ejemplo 6



Datos de entrada:

- Nota del estudiante (centinela)

Datos de salida:

- Porcentaje de estudiantes que ganaron
- Porcentaje de estudiantes que perdieron
- Promedio de notas de los que ganaron
- Promedio de notas de los que perdieron
- Promedio de notas del curso.

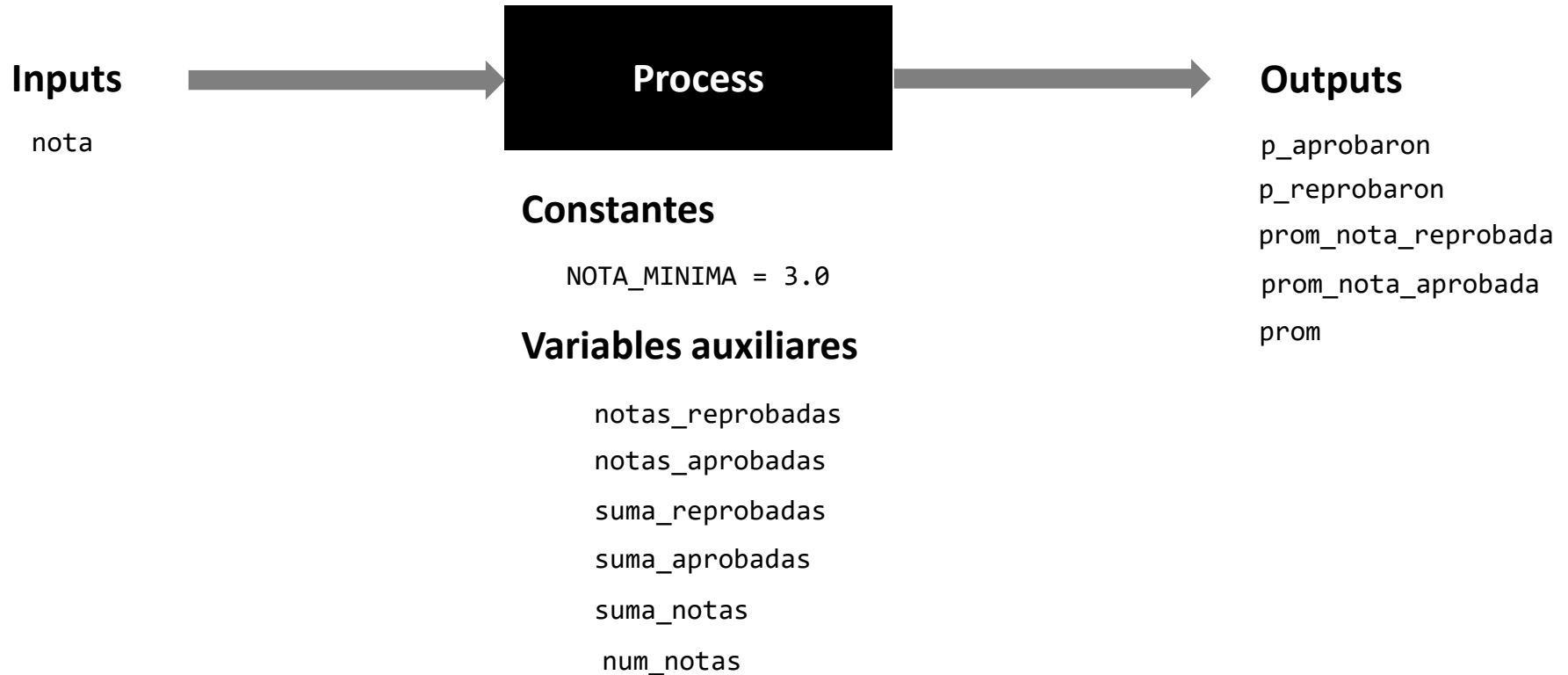
Definición de variables:

- **Entrada/salida:**
 - **nota:** Nota del estudiante (-1 significa que no hay mas notas).
 - **p_ganadores:** Porcentaje ganadores.
 - **p_perdedores:** Porcentaje perdedores.
 - **prom_nota_reprobada:** Promedio notas reprobadas.
 - **prom_nota_aprobada:** Promedio de notas aprobadas.
 - **prom:** Nota del estudiante

Definición de variables:

- **Variables auxiliares:**
 - **notas_reprobadas:** Numero de notas reprobadas (contador)
 - **suma_reprobadas:** Suma de notas reprobadas (acumulador).
 - **notas_aprobadas:** Numero de notas aprobadas (contador)
 - **suma_aprobadas:** Suma de notas reprobadas (acumulador).
 - **num_notas:** Numero de notas registradas.
 - **suma_notas:** Suma de las notas registradas.

Ciclos – Ejemplo 6



Ciclos – Ejemplo 6 - Centinela

Inicio

NOTA_MINIMA = 3.0

notas_reprobadas = 0

notas_aprobadas = 0

suma_reprobadas = 0

suma_aprobadas = 0

Leer(nota)

Mientras (hay_estudiante != -1) Haga

Si nota >= NOTA_MINIMA Entonces

notas_aprobadas = notas_aprobadas + 1

suma_aprobadas = suma_aprobadas + nota

Sino

notas_reprobadas = notas_reprobadas + 1

suma_reprobadas = suma_reprobadas + nota

Fin_Si

Leer(nota)

Fin_Mientras

num_notas = notas_reprobadas + notas_aprobadas

suma_notas = suma_reprobadas + suma_aprobadas

Si num_notas == 0 Entonces

Escribir("No hay estudiantes")

Sino

p_aprobaron = (notas_aprobadas/num_notas)*100

p_reprobaron = (notas_reprobadas/num_notas)*100

prom_nota_reprobada = suma_reprobadas/notas_reprobadas

prom_nota_aprobada = suma_aprobadas/notas_aprobadas

prom = suma_notas/num_notas

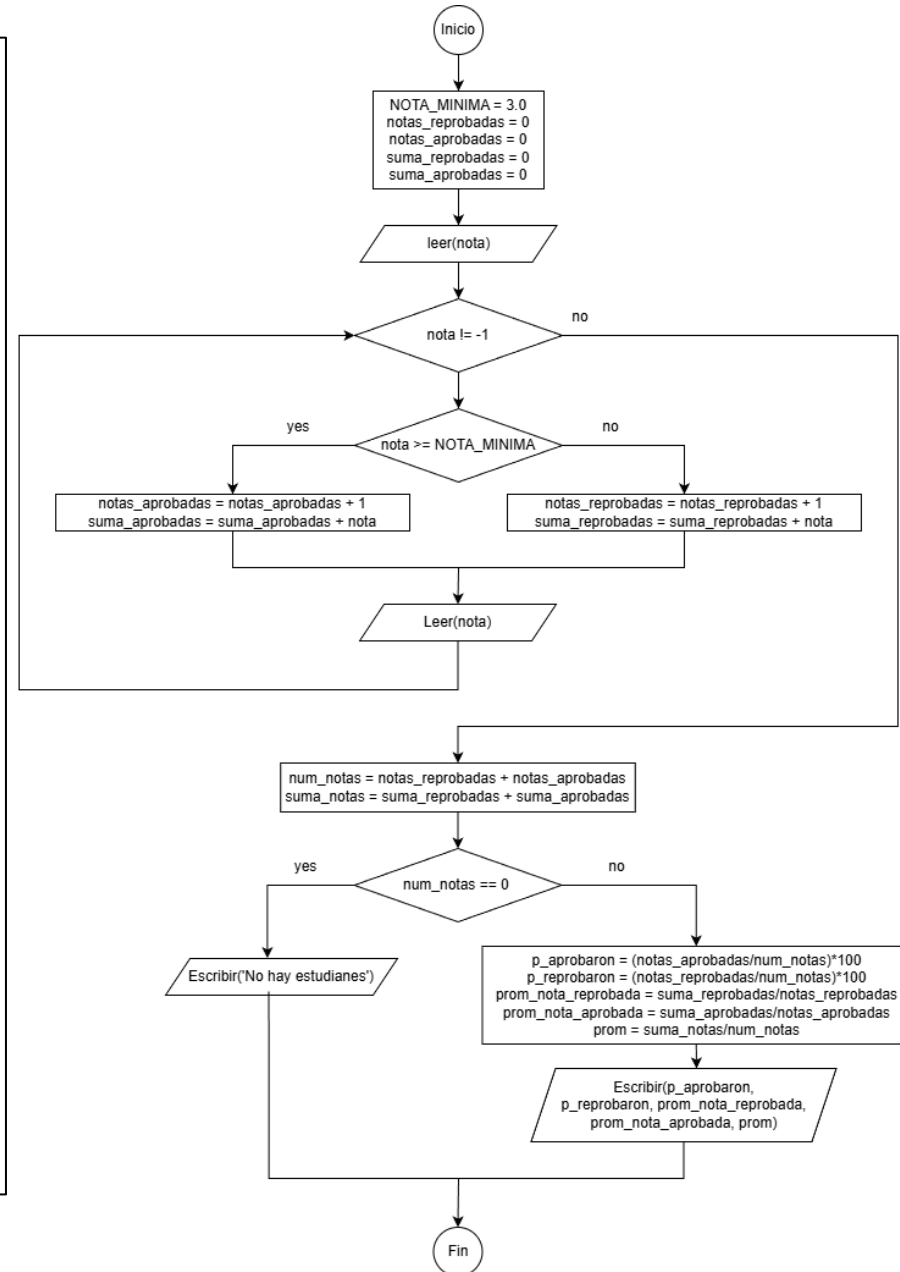
Escribir(p_aprobaron, p_reprobaron)

Escribir(prom_nota_reprobada, prom_nota_aprobada)

Escribir(prom)

Fin_Si

Fin



Ciclos – Ejemplo 6 - Centinela

Inicio

NOTA_MINIMA = 3.0

notas_reprobadas = 0

notas_aprobadas = 0

suma_reprobadas = 0

suma_aprobadas = 0

Leer(nota)

Mientras (hay_estudiante != -1) Haga

Si nota >= NOTA_MINIMA Entonces

notas_aprobadas = notas_aprobadas + 1

suma_aprobadas = suma_aprobadas + nota

Sino

notas_reprobadas = notas_reprobadas + 1

suma_reprobadas = suma_reprobadas + nota

Fin_Si

Leer(nota)

Fin_Mientras

num_notas = notas_reprobadas + notas_aprobadas

suma_notas = suma_reprobadas + suma_aprobadas

Si num_notas == 0 Entonces

Escribir("No hay estudiantes")

Sino

p_aprobaron = (notas_aprobadas/num_notas)*100

p_reprobaron = (notas_reprobadas/num_notas)*100

prom_nota_reprobada = suma_reprobadas/notas_reprobadas

prom_nota_aprobada = suma_aprobadas/notas_aprobadas

prom = suma_notas/num_notas

Escribir(p_aprobaron, p_reprobaron)

Escribir(prom_nota_reprobada, prom_nota_aprobada)

Escribir(prom)

Fin_Si

Fin

Resultados esperados

Notas	4.0, 2.5, 3.0, 1.5, 4.8
Aprobaron	(3) 60%
Reprobaron	(2) 40%
Promedio nota aprobada	3.93
Promedio nota reprobada	2.0
Promedio general	3.16

Prueba de escritorio

Realizar la prueba de escritorio para verificar que el algoritmo este bien planteado

Nota: Este programa así como esta planeado tiene aun un bug grave. Encuéntrelo y corríjalo.

Ciclos – Ejemplo 6 - Centinela

Resultados esperados

Notas	4.0, 2.5, 3.0, 1.5, 4.8
Aprobaron	(3) 60%
Reprobaron	(2) 40%
Promedio nota aprobada	3.93
Promedio nota reprobada	2.0
Promedio general	3.16

Forma

```
...  
Leer(nota)  
Mientras (nota != -1)  
Entonces  
    ...  
    Leer(nota)  
Fin_Mientras  
...
```

Resultado de ejecución

```
Ingrese la nota del estudiante (-1 para terminar): 4  
Ingrese la nota del estudiante (-1 para terminar): 2.5  
Ingrese la nota del estudiante (-1 para terminar): 3.0  
Ingrese la nota del estudiante (-1 para terminar): 1.5  
Ingrese la nota del estudiante (-1 para terminar): 4.8  
Ingrese la nota del estudiante (-1 para terminar): -1  
Porcentaje de estudiantes que aprobaron: 60.0%  
Porcentaje de estudiantes que reprobaron: 40.0%  
Promedio de notas reprobadas: 2.0  
Promedio de notas aprobadas: 3.9333333333333336  
Promedio general de notas: 3.16
```

Centinela

- Un centinela es un valor especial, mezclado dentro de los mismos datos de entrada, que se usa para indicar el fin de la lista de datos.
- Debe ser un valor que **esté fuera del rango válido de los datos reales**, para que nunca se confunda con un dato legítimo.
- Se usa directamente en la condición del ciclo, sin necesitar una variable adicional (a diferencia de la bandera).
- **Recordando el ejemplo 6:** como las notas van de 0.0 a 5.0, ningún valor real puede ser negativo (por eso -1 es un centinela seguro). El ciclo se controla directamente con la variable **nota**:

```
...  
Leer(nota)  
  Mientras (nota != -1) Haga  
    ...  
    Leer(nota)  
Fin_Mientras  
...
```

Ciclos – Ejemplo 7

Hacer un programa que lea primero el número de estudiantes de un curso (**N**), y luego lea las **N** notas correspondientes (valores decimales entre 0.0 y 5.0), una a una.

Considere que la nota mínima aprobatoria es 3.0: una nota es aprobada si es mayor o igual a 3.0, y reprobada si es menor a 3.0.

Al finalizar la lectura, el programa deberá entregar:

- El porcentaje de estudiantes aprobados.
- El porcentaje de estudiantes reprobados.
- El promedio de las notas aprobadas.
- El promedio de las notas reprobadas.
- El promedio de las notas del curso.

Ciclos – Ejemplo 7



Datos de entrada:

- Cantidad de estudiantes
- Nota del estudiante

Datos de salida:

- Porcentaje de estudiantes que ganaron
- Porcentaje de estudiantes que perdieron
- Promedio de notas de los que ganaron
- Promedio de notas de los que perdieron
- Promedio de notas del curso.

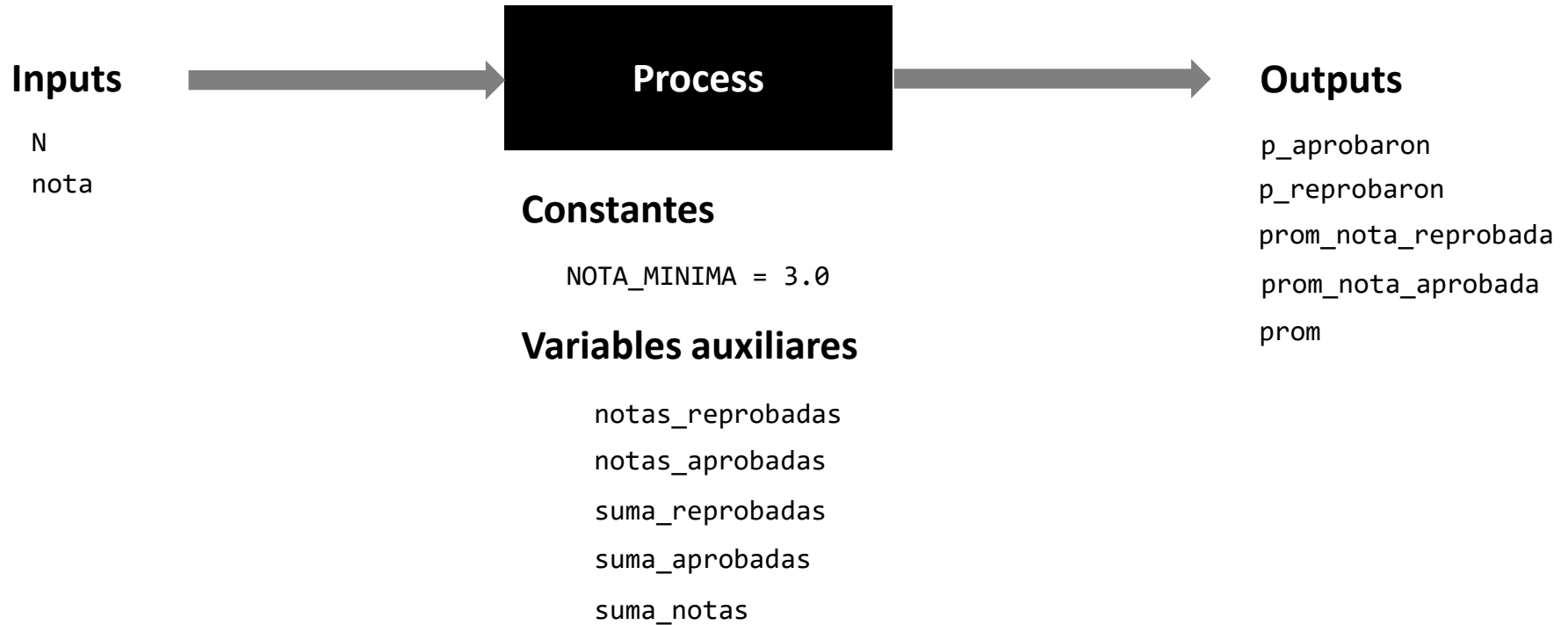
Definición de variables:

- **Entrada/salida:**
 - **N:** Numero de estudiantes
 - **nota:** Nota del estudiante (-1 significa que no hay mas notas).
 - **p_ganadores:** Porcentaje ganadores.
 - **p_perdedores:** Porcentaje perdedores.
 - **prom_nota_reprobada:** Promedio notas reprobadas.
 - **prom_nota_aprobada:** Promedio de notas aprobadas.
 - **prom:** Nota del estudiante

Definición de variables:

- **Variables auxiliares:**
 - **i:** Contador asociado a la i-esima nota
 - **notas_reprobadas:** Numero de notas reprobadas (contador)
 - **suma_reprobadas:** Suma de notas reprobadas (acumulador).
 - **notas_aprobadas:** Numero de notas aprobadas (contador)
 - **suma_aprobadas:** Suma de notas reprobadas (acumulador).
 - **suma_notas:** Suma de las notas registradas.

Ciclos – Ejemplo 7



Ciclos – Ejemplo 7

Inicio

NOTA_MINIMA = 3.0

notas_reprobadas = 0

notas_aprobadas = 0

suma_reprobadas = 0

suma_aprobadas = 0

i = 0

Leer(N)

Mientras (i < N) Haga

Leer(nota)

Si nota >= NOTA_MINIMA Entonces

notas_aprobadas = notas_aprobadas + 1

suma_aprobadas = suma_aprobadas + nota

Sino

notas_reprobadas = notas_reprobadas + 1

suma_reprobadas = suma_reprobadas + nota

Fin_Si

i = i + 1

Fin_Mientras

suma_notas = suma_reprobadas + suma_aprobadas

Si N == 0 Entonces

Escribir("No hay estudiantes")

Sino

p_aprobaron = (notas_aprobadas/N)*100

p_reprobaron = (notas_reprobadas/N)*100

prom_nota_reprobada = suma_reprobadas/notas_reprobadas

prom_nota_aprobada = suma_aprobadas/notas_aprobadas

prom = suma_notas/N

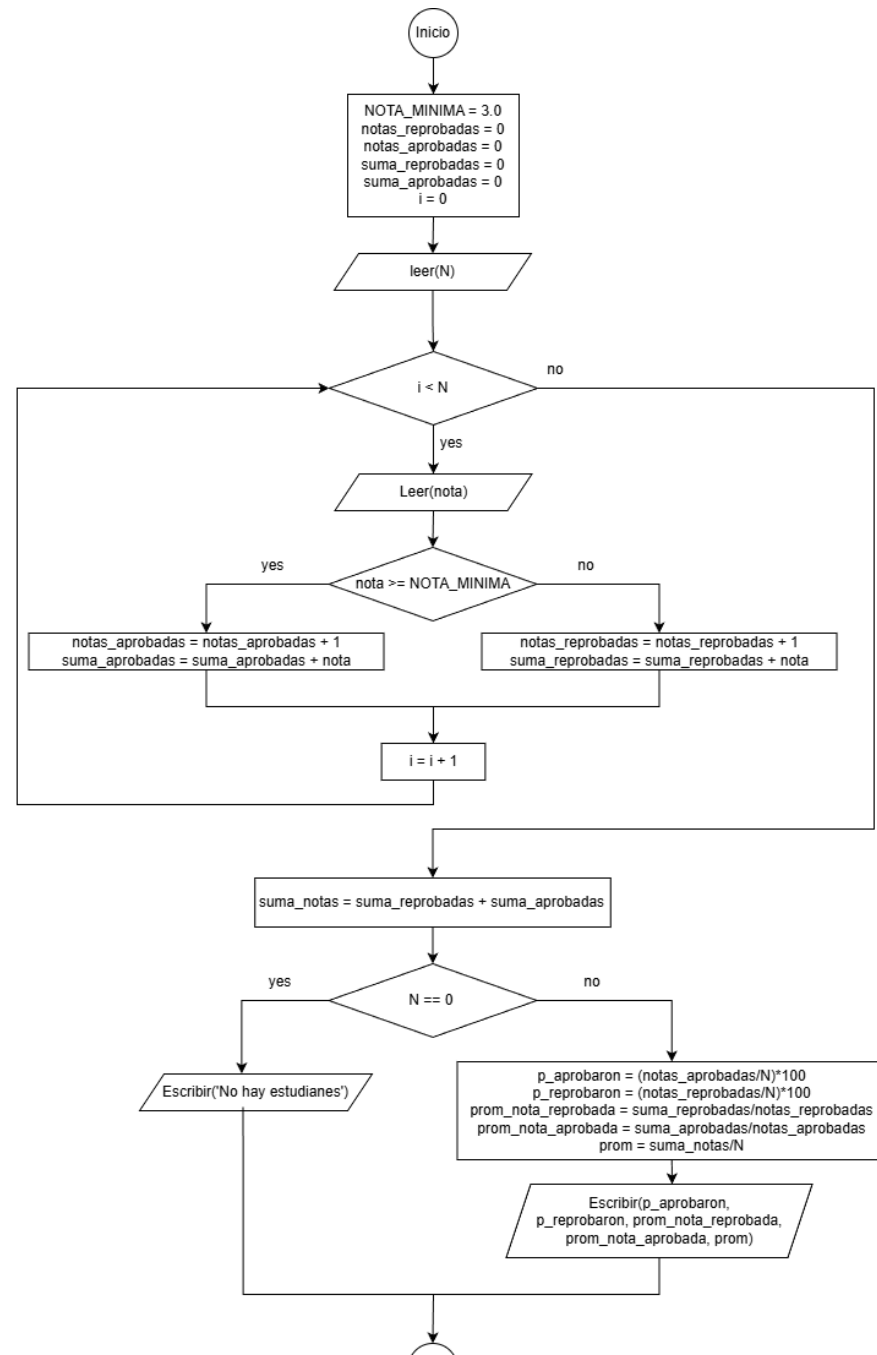
Escribir(p_aprobaron, p_reprobaron)

Escribir(prom_nota_reprobada, prom_nota_aprobada)

Escribir(prom)

Fin_Si

Fin



Ciclos – Ejemplo 7

Inicio

NOTA_MINIMA = 3.0

notas_reprobadas = 0

notas_aprobadas = 0

suma_reprobadas = 0

suma_aprobadas = 0

i = 0

Leer(N)

Mientras (i < N) Haga

Leer(nota)

Si nota >= NOTA_MINIMA Entonces

notas_aprobadas = notas_aprobadas + 1

suma_aprobadas = suma_aprobadas + nota

Sino

notas_reprobadas = notas_reprobadas + 1

suma_reprobadas = suma_reprobadas + nota

Fin_Si

i = i + 1

Fin_Mientras

suma_notas = suma_reprobadas + suma_aprobadas

Si N == 0 Entonces

Escribir("No hay estudiantes")

Sino

p_aprobaron = (notas_aprobadas/N)*100

p_reprobaron = (notas_reprobadas/N)*100

prom_nota_reprobada = suma_reprobadas/notas_reprobadas

prom_nota_aprobada = suma_aprobadas/notas_aprobadas

prom = suma_notas/N

Escribir(p_aprobaron, p_reprobaron)

Escribir(prom_nota_reprobada, prom_nota_aprobada)

Escribir(prom)

Fin_Si

Fin

Resultados esperados

Numero de estudiantes	5
Notas	4.0, 2.5, 3.0, 1.5, 4.8
Aprobaron	(3) 60%
Reprobaron	(2) 40%
Promedio nota aprobada	3.93
Promedio nota reprobada	2.0
Promedio general	3.16

Prueba de escritorio

Realizar la prueba de escritorio para verificar que el algoritmo este bien planteado

Nota: Este programa así como esta planeado tiene aun un bug grave. Encuéntrelo y corríjalo.

Ciclos – Ejemplo 7

Resultados esperados

Numero de estudiantes	5
Notas	4.0, 2.5, 3.0, 1.5, 4.8
Aprobaron	(3) 60%
Reprobaron	(2) 40%
Promedio nota aprobada	3.93
Promedio nota reprobada	2.0
Promedio general	3.16

Forma

```
i = 0
...
Mientras (i < N) Entonces
    Leer(nota)
    ...
    i = i + 1
Fin_Mientras
...
```

Resultado de ejecución

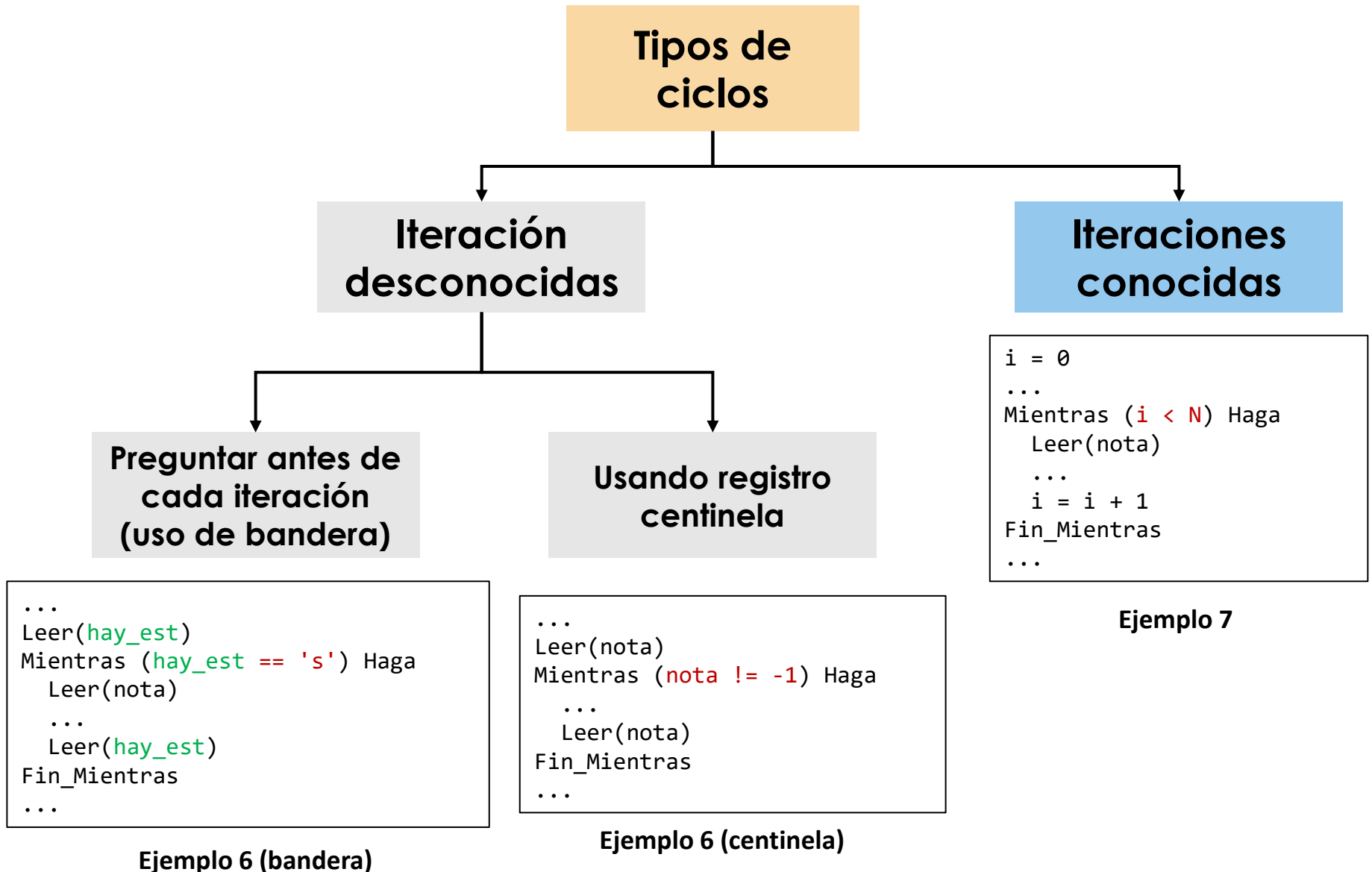
```
Ingrese el número de estudiantes: 5
Ingrese la nota del estudiante 1: 4
Ingrese la nota del estudiante 2: 2.5
Ingrese la nota del estudiante 3: 3.0
Ingrese la nota del estudiante 4: 1.5
Ingrese la nota del estudiante 5: 4.8
Porcentaje de estudiantes que aprobaron: 60.0%
Porcentaje de estudiantes que reprobaron: 40.0%
Promedio de notas reprobadas: 2.0
Promedio de notas aprobadas: 3.9333333333333336
Promedio general de notas: 3.16
```

Tipos de variables útiles en los ciclos

Todo ciclo suele apoyarse en algunos tipos de variables que cumplen roles específicos:

- **Variable de control:** se evalúa en la condición del ciclo para decidir si continúa o termina
- **Contador:** lleva la cuenta de repeticiones; se actualiza en una cantidad constante.
- **Acumulador:** acumula el resultado de una operación que se repite; se actualiza en un valor variable.
- **Bandera:** variable de dos valores excluyentes usada para controlar el ciclo o recordar un estado dentro de él.
- **Centinela:** valor especial, fuera del rango válido de los datos, que señala el fin de la entrada.

Recapitulando lo visto



Ciclos infinitos

- Un ciclo infinito es aquel cuya condición nunca se vuelve falsa por sí sola.
- Puede ser un error de programación (olvidó de la actualización de la variable de control) y hay que aprender a detectarlo.
- También puede ser intencional: cuando no existe una condición de parada natural para escribir desde el inicio del ciclo.
- Un ciclo infinito intencional solo funciona si existe una forma de detenerlo desde adentro (lo cual se resuelve mediante la ruptura de ciclos).

Ciclo infinito por error

```
i = 0
Mientras (i < 10) Haga
    Escribir(i)
    // Falta actualizar i -> el ciclo nunca termina
Fin_Mientras
```

Ciclo infinito intencional

```
Mientras (Verdadero) Haga
    Leer(dato)
    ...
Fin_Mientras
```

Ruptura de ciclos

A veces conviene detener un ciclo antes de que termine "por las buenas", es decir, antes de que su condición se vuelva falsa de forma natural. Dos situaciones típicas:

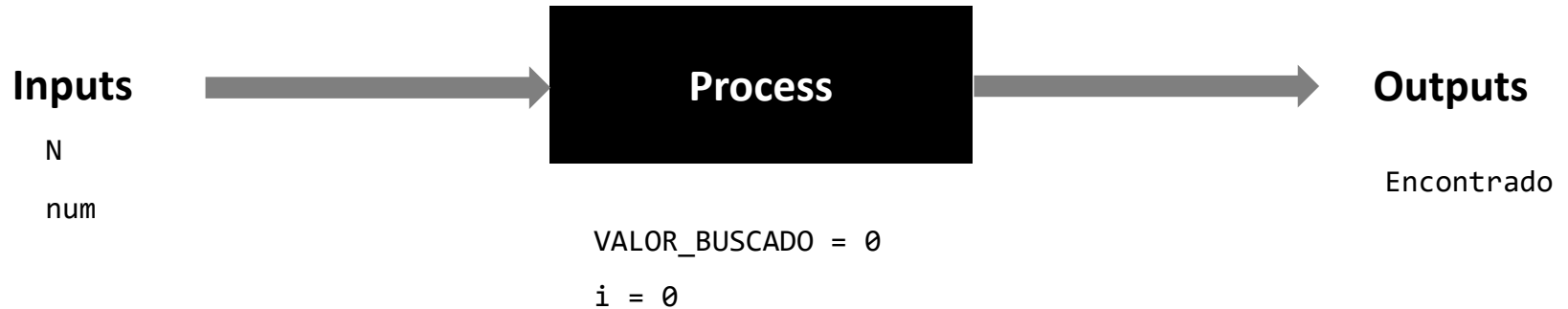
- Ya se cumplió lo que se estaba buscando (por ejemplo, ya se encontró el elemento buscado y no tiene sentido seguir recorriendo lo que falta).
- Ocurre una situación excepcional que impide completar el número de repeticiones previsto de forma normal (por ejemplo, un dato inválido o un error).

¿Cómo se logra la ruptura?

- **Forma estructurada:** se cambia el valor de la expresión lógica que controla el ciclo (por ejemplo, con una variable bandera) para que se vuelva falsa y el ciclo termine en su próxima evaluación.
- **Forma directa:** muchos lenguajes de programación ofrecen una instrucción especial, break, que interrumpe el ciclo de inmediato, sin esperar a que se reevalúe la condición.

Ruptura de ciclos – Ejemplo 8

Se tiene una serie de **N** números enteros. Se quiere saber si dentro de ellos existe, por lo menos, uno cuyo valor sea cero.



Datos de entrada:

- Cantidad de números
- Valor del numero ingresado

Datos de salida:

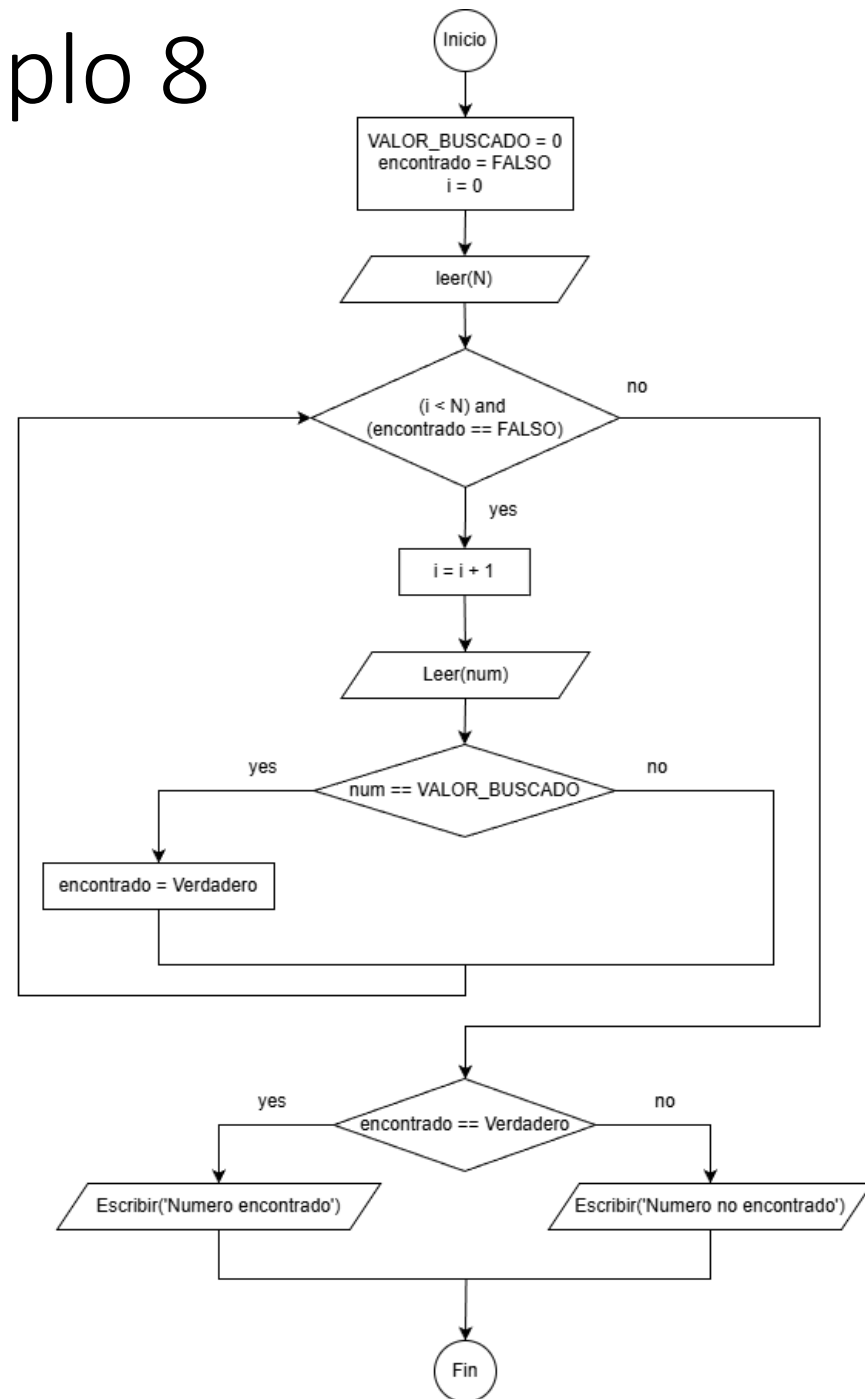
- Numero encontrado

Definición de variables:

- **N:** Cantidad de números
- **num:** Numero leído en la iteración i-esima
- **encontrado:** Bandera que indica que el número se encontró. Se inicializa en falso.
- **i:** Variable de control del ciclo (contador).

Ruptura de ciclos – Ejemplo 8

```
Inicio
  VALOR_BUSCADO = 0
  encontrado = Falso
  i = 0
  Leer(N)
  Mientras ((i < N) and (encontrado == Falso)) Haga
    Leer(num)
    i = i + 1
    Si num == VALOR_BUSCADO Entonces
      encontrado = Verdadero
    Fin_Si
  Fin_Mientras
  Si encontrado == Falso Entonces
    Escribir("Numero encontrado.")
  Sino
    Escribir("Numero no encontrado.")
  Fin_Si
Fin
```



Ruptura de ciclos – Ejemplo 8

```
Inicio
  VALOR_BUSCADO = 0
  encontrado = Falso
  i = 0
  Leer(N)
  Mientras ((i < N) and (encontrado == Falso)) Haga
    Leer(num)
    i = i + 1
    Si num == VALOR_BUSCADO Entonces
      encontrado = Verdadero
    Fin_Si
  Fin_Mientras
  Si encontrado == Verdadero Entonces
    Escribir("Numero encontrado.")
  Sino
    Escribir("Numero no encontrado.")
  Fin_Si
Fin
```

Resultados esperados

Cantidad de números	5
Números	-10, 15, 20, 8, 2
Encontrado	Falso

Cantidad de números	4
Números	0, 13, 1, 8
Encontrado	Verdadero

Prueba de escritorio

Realizar la prueba de escritorio para verificar que el algoritmo este bien planteado

Ruptura de ciclos – Ejemplo 8 (Comparación)

Las dos versiones resuelven el mismo problema y producen el mismo resultado.

- **Izquierda (forma estructurada):** el **Mientras** incluye **encontrado == Falso** en su condición — el ciclo se detiene solo, sin instrucciones adicionales en el cuerpo.
- **Derecha (con Romper):** el **Mientras** se simplifica a **i < N**; la salida anticipada queda a cargo de **Romper**, que interrumpe el ciclo apenas se cumple la condición interna, sin esperar a que se reevalúe el encabezado.
- **La condición de parada no desaparece (se mueve):** de estar visible en el encabezado del ciclo, pasa a estar escondida dentro del cuerpo.

```
Inicio
  VALOR_BUSCADO = 0
  encontrado = Falso
  i = 0
  Leer(N)
  Mientras ((i < N) and (encontrado == Falso)) Haga
    Leer(num)
    i = i + 1
    Si num == VALOR_BUSCADO Entonces
      encontrado = Verdadero
    Fin_Si
  Fin_Mientras
  Si encontrado == Verdadero Entonces
    Escribir("Numero encontrado.")
  Sino
    Escribir("Numero no encontrado.")
  Fin_Si
Fin
```

```
Inicio
  VALOR_BUSCADO = 0
  encontrado = Falso
  i = 0
  Leer(N)
  Mientras ((i < N)) Haga
    Leer(num)
    i = i + 1
    Si num == VALOR_BUSCADO Entonces
      encontrado = Verdadero
      Romper
    Fin_Si
  Fin_Mientras
  Si encontrado == Verdadero Entonces
    Escribir("Numero encontrado.")
  Sino
    Escribir("Numero no encontrado.")
  Fin_Si
Fin
```

Ruptura de ciclos – Ejemplo 8

- La implementación en Python de ambas soluciones en pseudocódigo se muestran a continuación.
- Note que el equivalente de la instrucción de nuestro pseudocódigo **Romper** en Python es **break**.

```
VALOR_BUSCADO = 0
encontrado = False
i = 0

N = int(input('Ingrese el número de números: '))
while ((i < N) and (not(encontrado))):
    i += 1    # i = i + 1
    num = int(input(f'Ingrese el número {i}: '))
    if num == VALOR_BUSCADO:
        encontrado = True

if encontrado:
    print(str(VALOR_BUSCADO) +
          " encontrado en la posición " +
          str(i))
else:
    print(str(VALOR_BUSCADO) + " no encontrado")
```

[Link simulación](#)

```
VALOR_BUSCADO = 0
encontrado = False
i = 0

N = int(input('Ingrese el número de números: '))
while (i < N):
    i += 1    # i = i + 1
    num = int(input(f'Ingrese el número {i}: '))
    if num == VALOR_BUSCADO:
        encontrado = True
        break # Instrucción de ruptura

if encontrado:
    print(str(VALOR_BUSCADO) +
          " encontrado en la posición " +
          str(i))
else:
    print(str(VALOR_BUSCADO) + " no encontrado")
```

[Link simulación](#)

Ruptura de ciclos – Casos de uso comunes

A continuación, veremos varias situaciones típicas donde se puede aplicar la ruptura de ciclos:

- **Validación de datos de entrada:** repetir hasta que el dato ingresado sea válido.
- **Menú interactivo:** repetir hasta que el usuario decida salir.
- **Lectura hasta un centinela:** procesar datos sin saber de antemano cuántos vendrán.
- **Detección de un error:** detener el ciclo apenas aparece un dato inválido.
- **Límite de intentos:** combinar dos motivos distintos para terminar (éxito o tope de intentos).
- **Ciclos anidados:** Romper solo corta el ciclo inmediato que lo contiene.

Validación de datos de entrada

- No se sabe cuántos intentos hará falta.
- **Romper** decide, desde adentro, cuándo el dato ya es válido.
- Evita inventar una condición inicial artificial en el **Mientras**.

```
Inicio
  Mientras (Verdadero) Haga
    Leer(edad)
    Si (edad >= 0) and (edad <= 120) Entonces
      Romper
    Sino
      Escribir("Edad invalida, intente de nuevo.")
    Fin_Si
  Fin_Mientras
  Escribir("Edad registrada: ", edad)
Fin
```

```
while True:
    edad = int(input('Ingrese su edad: '))
    if 0 <= edad <= 120:
        break # Instrucción de ruptura
    else:
        print("Edad invalida, intente de nuevo.")
print(f"Edad registrada: {edad}")
```

Menú interactivo

- No hay condición natural para el encabezado del ciclo.
- **Romper** corta justo donde el usuario elige "Salir".
- Las demás opciones no interrumpen el ciclo.

```
Inicio
  Mientras (Verdadero) Haga
    Escribir("1. Agregar")
    Escribir("2. Consultar")
    Escribir("3. Salir")
    Leer(opcion)
    Si opcion == 3 Entonces
      Romper
    Fin_Si
    Si opcion == 1 Entonces
      Escribir("Agregando...")
    Fin_Si
    Si opcion == 2 Entonces
      Escribir("Consultando...")
    Fin_Si
  Fin_Mientras
  Escribir("Programa finalizado.")
Fin
```

```
while True:
    print("1. Agregar")
    print("2. Consultar")
    print("3. Salir")
    opcion = int(input("Elija una opcion: "))

    if opcion == 3:
        break # Instrucción de ruptura
    elif opcion == 1:
        print("Agregando...")
    elif opcion == 2:
        print("Consultando...")
    else:
        print("Opcion invalida")

print("Programa finalizado.")
```

Lectura hasta un centinela

- No se sabe cuántos datos ingresará el usuario.
- **Romper** actúa apenas aparece el centinela (-1).
- Se rompe antes de sumarlo, para no contaminar el resultado.

```
Inicio
suma = 0
Mientras (Verdadero) Haga
    Leer(numero)
    Si numero == -1 Entonces
        Romper
    Fin_Si
    suma = suma + numero
Fin_Mientras
Escribir("Suma total: ", suma)
Fin
```

```
suma = 0
while True:
    numero = int(input('Ingrese un numero (-1 para
terminar): '))
    if numero == -1:
        break # Instrucción de ruptura
    suma = suma + numero

print(f"Suma total: {suma}")
```

Detección de una condición de error

- Un dato inválido invalida el resultado final.
- **Romper** corta de inmediato, sin esperar el resto del ciclo.
- La bandera solo registra que hubo error, no controla el ciclo.

```
Inicio
  Leer(N)
  i = 0
  huboError = Falso
  Mientras (i < N) Haga
    Leer(dato)
    Si dato < 0 Entonces
      Escribir("Dato invalido en la posicion ", i)
      huboError = Verdadero
      Romper
    Fin_Si
    i = i + 1
  Fin_Mientras
  Si huboError == Falso Entonces
    Escribir("Todos los datos se procesaron
correctamente.")
  Fin_Si
Fin
```

```
N = int(input('Cantidad de datos: '))
i = 0
huboError = False

while i < N:
    dato = int(input(f'Ingrese el dato {i}: '))
    if dato < 0:
        print(f"Dato invalido en la posicion {i}")
        huboError = True
        break # Instrucción de ruptura
    i += 1

if not huboError:
    print("Todos los datos se procesaron
correctamente.")
```

Limite de intentos

- Dos motivos de parada: éxito o agotar intentos.
- **Romper** controla el éxito (acertar la clave).
- El **Mientras** controla el fracaso (agotar intentos).

```
Inicio
  intentos = 0
  correcta = Falso
  Mientras (intentos < 3) Haga
    Leer(clave)
    Si clave == claveReal Entonces
      correcta = Verdadero
      Romper
    Fin_Si
    intentos = intentos + 1
    Escribir("Clave incorrecta, intento ",
intentos, " de 3")
  Fin_Mientras
  Si correcta == Verdadero Entonces
    Escribir("Acceso concedido.")
  Sino
    Escribir("Acceso bloqueado.")
  Fin_Si
Fin
```

```
intentos = 0
correcta = False
clave_real = "1234"

while intentos < 3:
    clave = input('Ingrese su clave: ')
    if clave == clave_real:
        correcta = True
        break # Instrucción de ruptura
    intentos += 1
    print(f"Clave incorrecta, intento {intentos} de
3")

if correcta:
    print("Acceso concedido.")
else:
    print("Acceso bloqueado.")
```

Ciclos anidados

- Un solo divisor basta para descartar que sea primo.
- **Romper** corta el ciclo interno de inmediato.
- El ciclo externo no se ve afectado.

```
Inicio
Leer(n)
numero = 2
Mientras (numero <= n) Haga
    esPrimo = Verdadero
    divisor = 2
    Mientras (divisor < numero) Haga
        Si numero % divisor == 0 Entonces
            esPrimo = Falso
            Romper
        Fin_Si
        divisor = divisor + 1
    Fin_Mientras
    Si esPrimo == Verdadero Entonces
        Escribir(numero, " es primo")
    Fin_Si
    numero = numero + 1
Fin_Mientras
Fin
```

```
n = int(input('Ingrese un limite: '))

for numero in range(2, n + 1):
    esPrimo = True
    divisor = 2
    while divisor < numero:
        if numero % divisor == 0:
            esPrimo = False
            break # Instrucción de ruptura (solo
corta el ciclo interno)
            divisor += 1

    if esPrimo:
        print(f"{numero} es primo")
```

Proxima clase

- Solución del ejemplo 5 (cine del señor Burns) aplicando los conceptos vistos en esta clase y los conceptos nuevos.
- Estructura Haga.
- Ejemplo serie Fibonacci.
- Obteniendo valores extremos en ciclos.
- Ciclos anidados.

Ejercicios de repaso

1. Corrija el bug del ejemplo de las notas de clase (ejemplo 7).
2. Para cada uno de los estudiantes de una universidad, donde cada uno cursa 4 materias se tienen los siguientes datos:
 - Código del estudiante.
 - Nota materia 1.
 - Nota materia 2.
 - Nota materia 3.
 - Nota materia 4.

Hacer un algoritmo que encuentre y despliegue para cada estudiante el promedio crédito.

3. En un supermercado un ama de casa pone en su carrito los artículos que va tomando de los estantes. La señora quiere asegurarse de que el cajero le cobre bien lo que ella ha comprado; por lo cual cada vez que ella compra un artículo distinto le coloca un código numérico mayor que 0, anota la cantidad de artículos iguales y su precio y, determina cuánto dinero gastará en este artículo; a esto le suma lo que iría gastando en los demás artículos, hasta que decide que ya tomó lo que necesitaba. Hacer un algoritmo que le muestre a la señora el código del artículo, la cantidad de artículos de cada especie, el importe total por artículo y el importe total de la compra.
4. Elabore un algoritmo que lea un número entero n y que imprima todos los múltiplos de 3 hasta n .

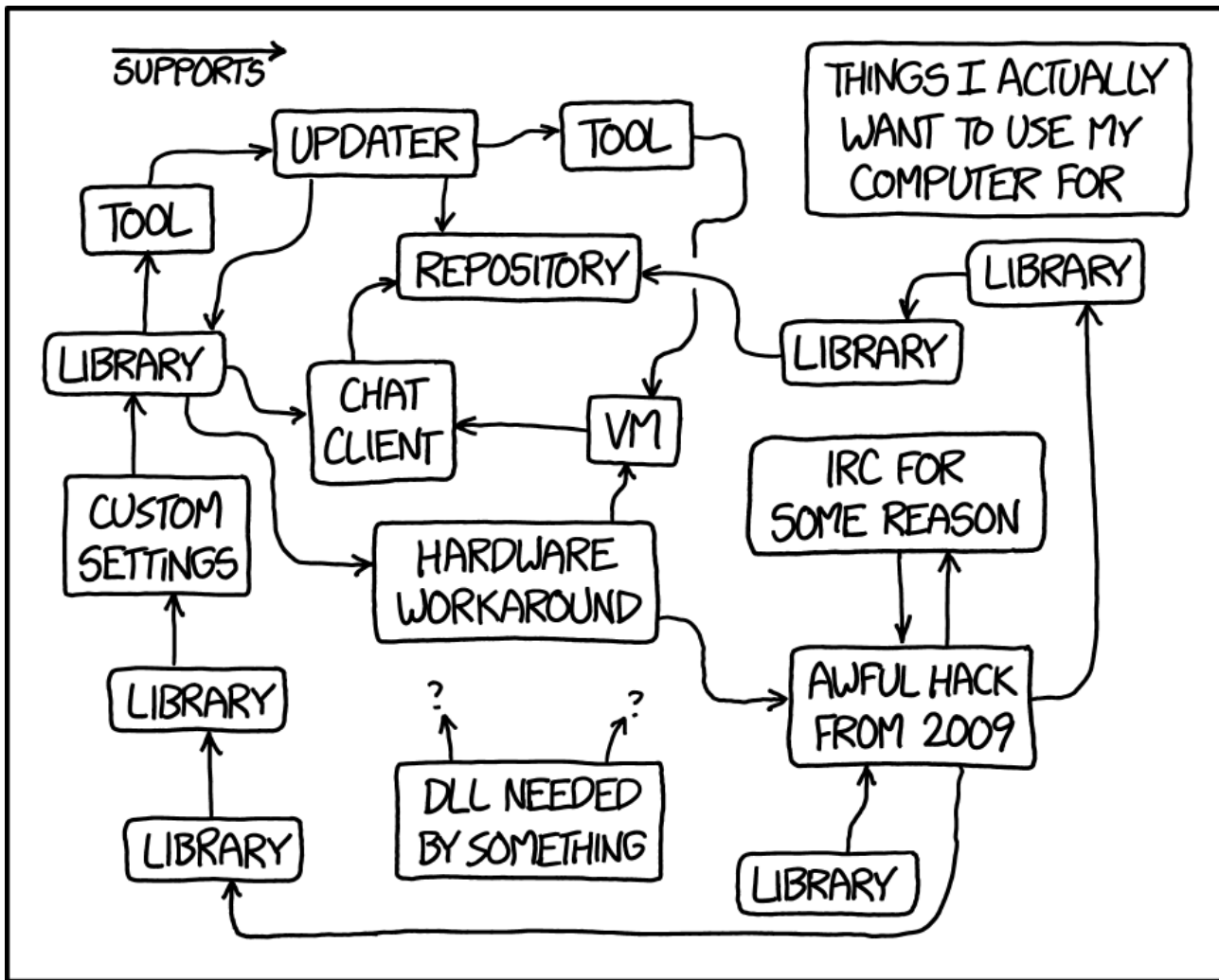
Ejercicios de repaso

4. Dados N valores, diseñe un algoritmo que haga el siguiente proceso:
 - Si el valor es menor que cero, calcular su cubo.
 - Si el valor esta entre 0 y 100, calcular su cuadrado.
 - Si el valor esta entre 101 y 1000 calcular su raíz cuadrada.
6. Imprimir los primeros N números pares, empezando en 2 (2, 4, 6, ..., hasta el N-ésimo par).
7. Dado N, imprimir los números desde N hasta 1 en orden descendente.
8. Imprimir la sucesión 1, 4, 9, 16, ..., hasta el cuadrado del N-ésimo término.
9. Imprimir 2, -4, 6, -8, 10, -12, ... hasta N términos.
10. Dado un número M y un rango [A, B], imprimir todos los múltiplos de M que estén entre A y B.
11. Leer N números y, después de cada uno, imprimir el total acumulado hasta ese momento (no solo el total final).
12. Leer números hasta que el usuario ingrese -1, e imprimir cada número junto con la posición en la que fue ingresado (1º, 2º, 3º...).

Referencias

1. <https://ellibrodepython.com/>
2. <https://www.w3resource.com/index.php>
3. <https://realpython.com/>
4. <https://curriculumresources.edu.gh/wp-content/uploads/2025/01/>
5. <https://python-course.eu/>
6. <https://realpython.com/cheatsheets/python/>
7. <https://realpython.com/python-conditional-statements/>
8. <https://realpython.com/ref/best-practices/conditionals/>
9. <https://makecode.microbit.org/courses/csintro>
10. <https://makecode.microbit.org/courses>
11. <https://realpython.com/python-while-loop/>
12. <https://realpython.com/ref/keywords/while/>

Eso es todo amigos



EVERY NOW AND THEN I REALIZE I'M MAINTAINING A HUGE CHAIN OF TECHNOLOGY SOLELY TO SUPPORT ITSELF.